

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра информатики

**РАЗРАБОТКА FULLSTACK WEB-ПРИЛОЖЕНИЯ ДЛЯ СОЗДАНИЯ И
АНАЛИЗА ФОРМ С ИСПОЛЬЗОВАНИЕМ ФРЕЙМВОРКОВ EF CORE
И BLAZOR**

(Выпускная квалификационная работа)

Выполнил студент 4.306-М3
группы
Коротких Дмитрий Андреевич

(подпись)

Научный руководитель:
к.ф.-м.н., доцент
Половикова Ольга Николаевна

(подпись)

Работа защищена
« ____ » _____ 2025г.
Оценка: _____
Председатель ГЭК: _____

Допустить к защите:
Зав. Кафедрой,
к.ф. - м.н., доцент
Козлов Денис Юрьевич

(подпись)

« ____ » _____ 2025г.

Барнаул 2025

РЕФЕРАТ

Тема выпускной работы: создание веб-приложения для создания шаблонов и обработки данных полученных форм.

Цель работы: спроектировать, реализовать и развернуть веб-приложение, позволяющее создавать, редактировать и публиковать шаблоны; собирать и анализировать полученные данные из заполненных форм.

Предмет исследования: система, позволяющую создавать формы, и анализировать полученные данные форм.

Объект исследования: проектировка, реализация и развертка облачных веб-приложений, решение задачи сбора и анализа данных пользователей.

Задачи исследования:

1. Исследовать существующие системы шаблонов и форм.
2. Проанализировать существующие подходы к разработке веб-приложений.
3. Проанализировать существующие фреймворки, библиотеки, решения хранения данных и сервисы облачного вычисления для создания веб-приложений.
4. Спроектировать веб-приложение.
5. Реализовать веб-приложение. Мигрировать базу данных. Отладить локальную версию.
6. Развернуть веб-приложение на облачном сервисе. Приобрести домен, настроить адресацию.

Объем работы: 62 страницы, количество рисунков – 4, таблиц – 0, приложений – 4, 27 использованных источников литературы.

Ключевые слова: ASP.NET Core, EF Core, Blazor, MudBlazor, SQLite, формы, шаблоны, анализ данных, веб-приложение, nginx.

ОГЛАВЛЕНИЕ

РЕФЕРАТ.....	1
ВВЕДЕНИЕ.....	4
Глава 1. Анализ существующих решений.....	7
1.1 Исследование существующих сервисов для создания и анализа форм.....	7
1.2 Анализ лучших практик пользовательского взаимодействия и социальных функций.....	13
Глава 2. Проектирование приложения.....	18
2.1 Анализ современных фреймворков, библиотек, и библиотек компонентов для реализации проекта.....	18
2.2 Проектировка работы приложения на высоком уровне.....	25
2.3 Проектировка структуры базы данных.....	27
2.4 Проектировка необходимых страниц.....	32
2.5 Проектировка необходимых ASP.NET Core сервисов.....	36
Глава 3. Реализация приложения.....	39
3.1 Реализация сущностей приложения. Миграция базы данных.....	39
3.2 Реализация сервисов приложения.....	39
3.3 Реализация razor страниц приложения.....	40
3.4 Реализация сервисов REST API, настройка API доступа.....	42
3.5 Доставка приложения на облачный сервис, конфигурация облачной машины. Настройка доменного имени.....	43
ЗАКЛЮЧЕНИЕ.....	45

БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....	47
Приложение 1.....	50
Приложение 2.....	52
Приложение 3.....	58
Приложение 4.....	61

ВВЕДЕНИЕ

Процессы сбора и анализа данных составляют основу информационной деятельности большинства организаций и предприятий. Онлайн-формы представляют собой один из наиболее эффективных инструментов для получения структурированной информации от пользователей, будь то обратная связь от клиентов, результаты опросов, заявки на участие в мероприятиях или регистрационные данные. Однако создание, управление и анализ таких форм часто требует значительных технических знаний или использования дорогостоящих специализированных платформ.

Существующие решения для создания форм, такие как Google Forms, Microsoft Forms или Typeform, представляют собой преимущественно инструментальные платформы, ориентированные на создание и распространение форм в рамках закрытых экосистем. Данные платформы имеют ряд функциональных ограничений: они не предоставляют возможности для публикации форм в общедоступном пространстве, не поддерживают функционал просмотра и использования публичных шаблонов других пользователей, ограничивают возможности социального взаимодействия и совместного использования контента. Кроме того, такие решения характеризуются недостаточной гибкостью в настройке интерфейса, ограниченными аналитическими возможностями и зависимостью от внешних сервисов. Многие организации и индивидуальные пользователи нуждаются в более комплексном решении, которое функционировало бы не только как инструмент создания форм, но и как платформа для публикации, обмена и совместного использования шаблонов с развитой системой аналитики

Актуальность данной работы обусловлена растущей потребностью в доступных и функциональных инструментах для создания и анализа онлайн-форм. Разработка собственного веб-приложения позволяет решить проблемы, связанные с ограничениями готовых решений, обеспечивая полный контроль над функционалом, данными и возможностями интеграции с другими системами организации.

Цель работы — спроектировать, реализовать и развернуть веб-приложение, позволяющее создавать, редактировать и публиковать шаблоны; собирать и анализировать полученные данные из заполненных форм.

Предмет исследования — система, позволяющую создавать формы, и анализировать полученные данные форм.

Объект исследования — проектировка, реализация и развертка облачных веб-приложений, решение задачи сбора и анализа данных пользователей.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих решений.
2. Изучить подходящие технологии для реализации проекта;
3. Спроектировать архитектуру приложения, включая модель данных, структуру сервисов и пользовательские интерфейсы;
4. Реализовать базу данных с использованием Entity Framework Core и системы миграций;
5. Реализовать серверные сервисы для обработки логики приложения;
6. Реализовать пользовательский интерфейс с использованием Blazor Razor-страниц;

7. Провести локальное тестирование и итеративную доработку функционала;
8. Развернуть приложение на облачном хостинге и протестировать его работу в production-среде.

Разработанное решение представляет собой интегрированную платформу, объединяющую функции создания форм, их публикации в общедоступном пространстве и детального анализа собранных данных с использованием современных технологий Microsoft .NET ecosystem.

Планируемое приложение будет иметь следующий функционал:

1. Базовый функционал управления аккаунтом — регистрация, логин, выход.
2. Редактор шаблонов — создание, редактирование шаблонов.
3. Инструменты администрирования — блокировка, удаление, мониторинг аккаунтов; управление ролями пользователей.
4. Сбор данных полученных форм. Анализ данных.
5. Каталог публичных шаблонов. Поиск, сортировка, фильтр шаблонов.
6. Социальные взаимодействия — лайки, комментарии.
7. Система техподдержки — интеграция с сервисом Jira.
8. Система дополнительных личных данных пользователей — интеграция с CRM сервисом Salesforce.

Глава 1. Анализ существующих решений.

1.1 Исследование существующих сервисов для создания и анализа форм

Для определения функциональных требований к разрабатываемому приложению необходимо провести анализ существующих решений в области создания и управления онлайн-формами. Современный рынок предлагает множество платформ различного уровня сложности и функциональности, от простых инструментов создания опросов до комплексных систем сбора и анализа данных. Исследование ключевых особенностей, преимуществ и ограничений наиболее популярных сервисов позволит выявить пробелы в функциональности существующих решений и обосновать необходимость разработки собственной платформы.

Google Forms представляет собой бесплатный веб-сервис для создания форм и опросов, интегрированный в экосистему Google Workspace. Платформа предоставляет базовые возможности для создания различных типов вопросов, включая текстовые поля, множественный выбор, шкалы оценки и загрузку файлов.

Преимущества:

- Простота использования и интуитивно понятный интерфейс
- Бесплатность для базового использования
- Автоматическая интеграция с Google Sheets для анализа данных
- Возможность совместного редактирования форм
- Широкий выбор шаблонов для быстрого создания форм
- Автоматическое сохранение данных в облачном хранилище

Недостатки:

- Ограниченные возможности кастомизации дизайна и брендинга

- Базовые аналитические функции без продвинутых инструментов анализа
- Зависимость от экосистемы Google и требование аккаунта Google
- Ограниченные возможности условной логики и сложных сценариев заполнения
- Отсутствие возможности публикации форм в общедоступном каталоге для просмотра и повторного использования другими пользователями
- Невозможность просмотра и адаптации публичных шаблонов, созданных сообществом пользователей

Microsoft Forms является частью пакета Microsoft 365 и предназначен для создания опросов, викторин и форм обратной связи. Сервис ориентирован на корпоративную среду и образовательные учреждения.

Преимущества:

- Интеграция с экосистемой Microsoft 365 и Teams
- Возможность создания интерактивных викторин с автоматической оценкой
- Продвинутое функции анализа данных через Power BI
- Поддержка совместной работы и делегирования прав доступа
- Встроенные шаблоны для различных сценариев использования
- Соответствие корпоративным стандартам безопасности и конфиденциальности

Недостатки:

- Требуется подписка на Microsoft 365 для полного функционала
- Ограниченные возможности визуальной настройки
- Привязка к корпоративной экосистеме Microsoft
- Сложность использования для пользователей, не знакомых с продуктами Microsoft
- Отсутствие механизма публикации форм в открытом доступе для использования сообществом
- Невозможность создания публичной библиотеки шаблонов для обмена между пользователями

Typeform позиционируется как платформа для создания интерактивных форм с акцентом на пользовательский опыт. Сервис отличается современным дизайном и возможностями создания conversational-форм, которые представляют информацию и вопросы пользователю последовательно, имитируя естественный диалог между человеком и системой. В отличие от традиционных статических форм, где все поля отображаются одновременно, conversational-формы показывают по одному вопросу за раз, адаптируя последующие вопросы на основе предыдущих ответов пользователя.

Преимущества:

- Высокое качество пользовательского интерфейса и UX
- Возможности создания интерактивных, пошаговых форм
- Широкие возможности кастомизации дизайна и брендинга
- Продвинутая условная логика и персонализация контента

- Интеграция с множеством внешних сервисов через API
- Детальная аналитика поведения пользователей при заполнении форм

Недостатки:

- Высокая стоимость премиум-планов для расширенного функционала
- Ограничения бесплатного тарифа по количеству ответов и форм
- Сложность настройки для пользователей без технических навыков
- Зависимость от внешней платформы для хранения и обработки данных
- Отсутствие возможности размещения созданных форм в публичной галерее для доступа других пользователей
- Невозможность использования и модификации публичных шаблонов, созданных сообществом

JotForm представляет собой комплексную платформу для создания форм с расширенными возможностями интеграции и автоматизации бизнес-процессов.

Преимущества:

- Обширная библиотека готовых шаблонов форм
- Возможности интеграции с CRM и системами управления документооборотом
- Поддержка электронных подписей и платежных систем
- Конструктор форм с drag-and-drop интерфейсом
- Возможности создания многостраничных форм со сложной логикой
- Встроенные инструменты для создания отчетов и дашбордов

Недостатки:

- Ограничения бесплатного тарифа по функциональности и количеству форм
- Сложность интерфейса для новых пользователей
- Потенциальные проблемы с производительностью при работе с большими формами
- Зависимость от внешних сервисов для расширенных интеграций
- Отсутствие механизма публичного обмена шаблонами между пользователями платформы
- Невозможность просмотра и использования форм, созданных другими пользователями сервиса

SurveyMonkey специализируется на создании опросов и исследований с акцентом на статистический анализ полученных данных.

Преимущества:

- Продвинутые инструменты статистического анализа и визуализации данных
- Широкие возможности сегментации и фильтрации респондентов
- Поддержка A/B тестирования опросов
- Экспорт данных в различные форматы для дальнейшего анализа
- Возможности проведения лонгитюдных исследований
- Соответствие стандартам проведения научных исследований

Недостатки:

- Высокая стоимость для доступа к продвинутым аналитическим функциям
- Фокус преимущественно на опросах, ограниченность в создании других типов форм
- Сложность настройки для простых сценариев использования
- Ограниченные возможности кастомизации внешнего вида
- Отсутствие возможности создания публичного каталога опросов для повторного использования
- Невозможность доступа к шаблонам опросов, созданным другими исследователями

Проведенный анализ показывает, что существующие платформы представляют собой преимущественно изолированные инструменты для создания форм, ориентированные на решение специфических задач в рамках закрытых экосистем. Основным общим недостатком всех рассмотренных сервисов является отсутствие функциональности социальной платформы, позволяющей пользователям публиковать созданные шаблоны в общедоступном пространстве и просматривать работы других авторов.

Данное ограничение препятствует формированию сообщества пользователей, обмену опытом и повторному использованию качественных решений. Более того, отсутствие социальных механизмов лишает исследователей и аналитиков дополнительного источника метаданных о качестве и эффективности созданных форм.

Интеграция социальных функций в систему управления формами открывает возможности для сбора и анализа метрик пользовательского

взаимодействия, которые могут служить индикаторами качества и релевантности контента. Выдвигается гипотеза о том, что социальные метрики (количество просмотров, оценки пользователей, комментарии) коррелируют с качеством собираемых данных и могут использоваться для валидации эффективности форм.

Например, низкий уровень пользовательской активности (малое количество положительных оценок, отсутствие комментариев, высокий процент частичного заполнения) может сигнализировать о проблемах в структуре или содержании формы, что, в свою очередь, ставит под сомнение репрезентативность и качество собранных данных. Анализ текстовых комментариев пользователей может выявить конкретные проблемы восприятия формы, такие как неясность формулировок, избыточная сложность или нерелевантность вопросов целевой аудитории.

Подобный подход к оценке качества форм на основе анализа социального взаимодействия представляет собой инновационный метод контроля качества данных, отсутствующий в существующих решениях. Разрабатываемая система должна устранить выявленные ограничения существующих платформ, интегрировав функции создания и анализа форм с механизмами публикации, совместного использования контента и соответствующими аналитическими инструментами для оценки качества на основе социальных метрик.

1.2 Анализ лучших практик пользовательского взаимодействия и социальных функций

На основе проведенного исследования существующих платформ для создания форм и анализа успешных социальных платформ необходимо выделить ключевые принципы и механизмы, которые могут быть адаптированы для разрабатываемой системы. Интеграция социальных

функций в приложение для создания форм требует понимания того, как пользователи взаимодействуют с контентом в различных цифровых сообществах.

Анализ успешных платформ показывает, что **низкий порог входа** является критически важным фактором для привлечения пользователей. Google Forms демонстрирует эффективность минималистичного подхода к созданию форм, где пользователь может создать базовую форму за несколько кликов. Typeform развивает этот принцип, предлагая интуитивно понятный drag-and-drop интерфейс с предварительным просмотром результата в реальном времени.

JotForm и Microsoft Forms успешно используют обширные **библиотеки** готовых шаблонов, что значительно ускоряет процесс создания форм. Однако их подход ограничивается статическим каталогом, созданным разработчиками платформы, без возможности пользователей создавать и делиться собственными решениями.

SurveyMonkey и Microsoft Forms демонстрируют важность встроенных **аналитических инструментов**, позволяющих пользователям получать инсайты из собранных данных без необходимости экспорта в внешние системы. Визуализация данных в реальном времени повышает вовлеченность пользователей и ценность платформы.

Reddit демонстрирует эффективность системы upvote/downvote, где пользователи могут **оценивать** контент, что влияет на его видимость и определяет **популярность** материалов. Подобные системы репутации поощряют пользователей создавать качественный контент и обеспечивают механизм самомодерации сообщества.

ArtStation использует комбинацию лайков, просмотров и комментариев для оценки популярности работ. Платформа подчеркивает важность релевантных тегов и описательных подписей для увеличения видимости контента, что может быть адаптировано для системы форм через теги категорий и описания назначения формы.

Reddit использует принцип кросс-постинга, позволяя размещать контент в нескольких релевантных сообществах, что **увеличивает охват** аудитории. Для системы форм это может быть реализовано через систему категорий и тегов, позволяющую одной форме быть видимой в нескольких тематических разделах. ArtStation применяет алгоритмы **рекомендаций** на основе интересов пользователя, истории просмотров и взаимодействий с контентом. Подобный подход может быть применен для предложения релевантных шаблонов форм на основе предыдущих действий пользователя.

Платформы для творческого контента, такие как Dribbble, подчеркивают важность конструктивной критики и **обратной связи** для улучшения качества работ. В контексте форм комментарии пользователей могут содержать ценную информацию о удобстве использования, ясности вопросов и общей эффективности формы. Reddit использует принцип, где upvotes показывают, что контент позитивно влияет на сообщество, а downvotes сигнализируют о неподходящем материале. Этот механизм может быть адаптирован для автоматического понижения рейтинга некачественных или нерелевантных форм.

Успешные платформы используют комбинацию пользовательских тегов и системных категорий для структурирования контента. ArtStation применяет иерархическую систему категорий в сочетании с свободными тегами, что обеспечивает как структурированную навигацию, так и гибкость в описании контента. Интеграция этих принципов в систему управления

формами позволяет создать дополнительный уровень валидации качества контента. Формы с высоким рейтингом пользователей и большим количеством положительных комментариев могут рассматриваться как более надежные источники данных.

Применение элементов геймификации, таких как система достижений за создание популярных форм, рейтинги авторов и значки экспертизы в определенных областях, может повысить мотивацию пользователей к созданию качественного контента. Использование данных о взаимодействии пользователей с различными типами форм для создания персонализированных рекомендаций может значительно улучшить пользовательский опыт и увеличить вероятность использования релевантных шаблонов.

Анализ лучших практик показывает, что успешная интеграция социальных функций в систему управления формами требует сбалансированного подхода между простотой создания контента и богатством социальных взаимодействий. Реализация выявленных принципов в разрабатываемой системе позволит создать платформу, которая будет не только инструментом для создания форм, но и активным сообществом пользователей, способствующим повышению качества и разнообразия доступного контента.

Ключевые элементы системы для реализации:

- Главная страница с пользовательским контентом вместо статической посадочной страницы;
- Система оценивания с моментальным визуальным откликом для мгновенной реакции пользователей;

- Интерфейс создания форм методом перетаскивания (drag-and-drop) с предварительным просмотром в реальном времени;
- Табличное представление данных для структурированной презентации собранной информации;
- Навигационные панели для быстрого доступа к категориям и фильтрам контента;
- Облако тегов для визуального представления популярных тем и упрощения навигации;
- Встроенные аналитические инструменты с визуализацией данных без необходимости экспорта;
- Система рекомендаций на основе пользовательского поведения и предпочтений;
- Элементы геймификации через систему достижений, рейтинги и экспертные значки

Глава 2. Проектирование приложения

2.1 Анализ современных фреймворков, библиотек, и библиотек компонентов для реализации проекта

Выбор технологического стека является критически важным этапом в процессе разработки веб-приложений, поскольку определяет архитектурные решения, временные затраты на реализацию и долгосрочную поддержку проекта. При выборе технологий для данного проекта приоритетными критериями являются: минимизация сложности реализации, обеспечение эффективного процесса отладки и сопровождения кода, а также использование языка программирования C#, с которым у разработчика имеется наибольший практический опыт. Данные факторы обуславливают необходимость детального анализа существующих решений с целью выявления оптимального набора технологий для достижения поставленных задач. Современная архитектура веб-приложений предполагает разделение на несколько ключевых уровней.

Фронтенд-технологии отвечают за создание пользовательского интерфейса и обеспечение взаимодействия с конечными пользователями, включая обработку пользовательского ввода, отображение данных и навигацию.

Бэкенд-фреймворки обеспечивают серверную логику приложения, обработку запросов, управление бизнес-процессами и интеграцию с внешними сервисами.

Системы управления базами данных предназначены для хранения, организации и извлечения структурированной информации, обеспечивая целостность и безопасность данных.

Библиотеки компонентов пользовательского интерфейса предоставляют готовые элементы интерфейса, такие как кнопки, формы,

таблицы и диалоговые окна, что существенно ускоряет процесс создания согласованного и функционального пользовательского интерфейса.

Анализ фреймворков фронтенда

React представляет собой библиотеку JavaScript для создания пользовательских интерфейсов, разработанную компанией Meta. Данная технология основана на компонентном подходе и использует виртуальную объектную модель документа (Virtual DOM) для оптимизации производительности. React обладает обширной экосистемой сторонних библиотек и инструментов, что обеспечивает гибкость в разработке. Однако использование React требует изучения дополнительных технологий, таких как системы управления состоянием (Redux, Context API) и инструменты сборки (Webpack, Vite).

Vue.js является прогрессивным фреймворком для создания пользовательских интерфейсов, который характеризуется постепенной кривой обучения и простотой интеграции в существующие проекты. Фреймворк предоставляет встроенную систему реактивности и двустороннюю привязку данных. Vue.js поддерживает как шаблонную разработку, так и использование функций рендеринга. Несмотря на свои преимущества, Vue.js имеет меньшую экосистему по сравнению с React и может потребовать дополнительных решений для сложных приложений.

ASP.NET Core Blazor представляет собой фреймворк для создания интерактивных веб-приложений с использованием языка программирования C# вместо JavaScript. Blazor позволяет разработчикам создавать как серверные (Blazor Server), так и клиентские (Blazor WebAssembly) приложения. Ключевым преимуществом данного решения является возможность использования единого языка программирования как для фронтенда, так и для бэкенда, что значительно упрощает процесс разработки.

и отладки. Blazor обеспечивает тесную интеграцию с экосистемой Microsoft .NET, включая Entity Framework Core и Visual Studio, что существенно облегчает навигацию по проекту и разработку полнофункциональных приложений. Полносвязная природа ASP.NET Core как единого фреймворка для фронтенда и бэкенда делает процесс отладки исключительно простым, поскольку разработчик может отслеживать выполнение кода от пользовательского интерфейса до базы данных в рамках одной среды разработки.

Анализ фреймворков бэкенда

Node.js представляет среду выполнения JavaScript на стороне сервера, построенную на движке V8. Express.js является минималистичным и гибким фреймворком для Node.js, предоставляющим основные функции для создания веб-приложений и программных интерфейсов приложений. Данное решение характеризуется высокой производительностью при обработке асинхронных операций и обширной экосистемой пакетов npm. Однако Node.js может испытывать затруднения при выполнении ресурсоёмких вычислений и требует дополнительных инструментов для обеспечения типобезопасности.

Django представляет собой высокоуровневый фреймворк для разработки веб-приложений на языке Python, предоставляющий полный набор встроенных компонентов и инструментов для разработки. Фреймворк предоставляет встроенную систему администрирования, объектно-реляционное отображение и систему аутентификации. Django обеспечивает высокий уровень безопасности благодаря встроенным механизмам защиты от распространённых уязвимостей. Недостатком данного решения может быть относительно медленная производительность по сравнению с компилируемыми языками и жёсткая структура проекта.

ASP.NET Core является кроссплатформенным фреймворком с открытым исходным кодом для создания современных веб-приложений и сервисов. Данная технология обеспечивает высокую производительность благодаря компилируемой природе языка C# и оптимизированной среде выполнения .NET. ASP.NET Core предоставляет встроенную поддержку внедрения зависимостей, middleware-компонентов и современных паттернов разработки. Существенным преимуществом является возможность создания полнофункциональных приложений с единым стек технологий, где серверная и клиентская части разрабатываются на одном языке программирования. Это обеспечивает упрощённый процесс разработки, отладки и сопровождения кода. Фреймворк легко компилируется для операционных систем Linux, что делает его оптимальным выбором для развёртывания на облачных серверах.

Анализ систем управления базами данных

PostgreSQL представляет собой объектно-реляционную систему управления базами данных с открытым исходным кодом, известную своей надёжностью и соответствием стандартам SQL. Система поддерживает сложные типы данных, включая JSON, массивы и пользовательские типы. PostgreSQL обеспечивает высокую производительность при работе с большими объёмами данных и поддерживает горизонтальное масштабирование. Однако данная система может требовать значительных ресурсов для небольших проектов и сложной настройки для оптимальной производительности.

MySQL является одной из наиболее популярных реляционных систем управления базами данных с открытым исходным кодом. Система характеризуется простотой использования, хорошей документацией и широкой поддержкой в различных средах разработки. MySQL обеспечивает

высокую скорость выполнения запросов и поддерживает репликацию данных. К недостаткам можно отнести ограниченную поддержку некоторых функций SQL и потенциальные проблемы с целостностью данных в определённых конфигурациях.

SQLite представляет собой встраиваемую реляционную базу данных, которая хранит всю информацию в одном файле. Данное решение характеризуется исключительной простотой развёртывания, поскольку не требует установки и настройки сервера базы данных. SQLite обеспечивает полную портативность данных, что существенно упрощает процесс переноса приложения между различными средами. База данных демонстрирует высокую производительность для приложений с умеренной нагрузкой и превосходно интегрируется с ASP.NET Core через Entity Framework Core. Компактность и автономность SQLite делают её оптимальным выбором для прототипирования и развёртывания приложений на облачных серверах, где важна простота администрирования и минимальные требования к конфигурации инфраструктуры.

Анализ библиотек компонентов пользовательского интерфейса

Material-UI представляет собой библиотеку компонентов React, реализующую принципы Material Design от Google. Библиотека предоставляет обширный набор готовых компонентов с современным дизайном и встроенными функциями доступности. Material-UI поддерживает темизацию и кастомизацию стилей. Однако использование данной библиотеки ограничено экосистемой React и может потребовать значительных усилий для кастомизации под специфические требования дизайна.

Bootstrap является одним из наиболее популярных CSS-фреймворков для создания адаптивных веб-интерфейсов. Фреймворк предоставляет сетку

компоновки, компоненты пользовательского интерфейса и утилиты для быстрой разработки. Bootstrap характеризуется широкой поддержкой браузеров и обширной документацией. К недостаткам можно отнести потенциальную похожесть интерфейсов, созданных с использованием стандартных тем, и необходимость дополнительной кастомизации для уникального дизайна.

MudBlazor представляет собой библиотеку компонентов пользовательского интерфейса, специально разработанную для ASP.NET Core Blazor. Библиотека предоставляет обширный набор современных интерактивных компонентов, реализованных в соответствии с принципами Material Design. MudBlazor обеспечивает нативную интеграцию с экосистемой Blazor, что гарантирует оптимальную производительность и совместимость. Компоненты библиотеки поддерживают двустороннюю привязку данных, валидацию форм и темизацию. Ключевым преимуществом MudBlazor является специализация на технологии Blazor, что обеспечивает безупречную интеграцию с серверными и клиентскими приложениями Blazor без необходимости дополнительных адаптеров или прослоек совместимости.

Анализ платформ облачного хостинга

Microsoft Azure представляет собой комплексную облачную платформу, предоставляющую широкий спектр сервисов для развёртывания и управления приложениями. Платформа обеспечивает нативную интеграцию с технологиями Microsoft .NET и предоставляет специализированные сервисы для ASP.NET Core приложений, включая Azure App Service. Azure характеризуется высокой надёжностью, масштабируемостью и развитой системой мониторинга. Однако использование Azure может быть экономически нецелесообразным для

небольших проектов и прототипов из-за относительно высокой стоимости услуг.

Amazon Web Services является одной из наиболее популярных облачных платформ, предоставляющей обширную экосистему сервисов для различных типов приложений. AWS поддерживает развёртывание .NET приложений через сервисы Elastic Beanstalk и EC2, обеспечивая гибкость в настройке инфраструктуры. Платформа характеризуется высокой производительностью и глобальной доступностью. К недостаткам можно отнести сложность ценообразования и потенциально высокие затраты при превышении лимитов бесплатного уровня.

Oracle Cloud Infrastructure представляет собой современную облачную платформу, предоставляющую вычислительные ресурсы и сервисы инфраструктуры. Ключевым преимуществом Oracle Cloud для задач прототипирования является предоставление бесплатного уровня Always Free, включающего две виртуальные машины с объёмом оперативной памяти 1 ГБ каждая, что обеспечивает достаточные ресурсы для развёртывания и тестирования прототипа приложения. Платформа поддерживает полную конфигурацию виртуальных машин, позволяя разработчику осуществлять детальную настройку операционной системы, сетевых параметров и программного обеспечения в соответствии с требованиями проекта. Данные возможности делают Oracle Cloud Infrastructure оптимальным выбором для разработки и тестирования прототипов без финансовых затрат на инфраструктуру.

На основании проведённого сравнительного анализа современных технологий разработки веб-приложений и платформ облачного хостинга для реализации данного проекта выбран следующий технологический стек: ASP.NET Core в качестве бэкенд-фреймворка, Blazor для создания

пользовательского интерфейса, SQLite как система управления базой данных, MudBlazor в качестве библиотеки компонентов интерфейса и Oracle Cloud Infrastructure для развёртывания приложения.

Данный выбор обусловлен соответствием указанных технологий установленным критериям отбора. Использование единой экосистемы Microsoft .NET обеспечивает минимизацию сложности реализации за счёт унифицированного подхода к разработке серверной и клиентской частей приложения. Применение языка программирования C# на всех уровнях архитектуры существенно упрощает процесс отладки и сопровождения кода, позволяя разработчику эффективно использовать накопленный практический опыт. Портативность SQLite и нативная интеграция всех выбранных компонентов в рамках единой технологической платформы создают оптимальные условия для быстрого прототипирования и последующего развёртывания приложения в производственной среде.

2.2 Проектировка работы приложения на высоком уровне.

Архитектура разработанного веб-приложения базируется на фреймворке ASP.NET Core с использованием технологии Blazor Server для создания интерактивного пользовательского интерфейса. Центральным элементом конфигурации системы выступает файл Program.cs, который отвечает за инициализацию, настройку внедрения зависимостей, построение и запуск веб-приложения.

В рамках архитектуры приложения реализована система внедрения зависимостей ASP.NET Core, основанная на концепции сервисов. Сервисы представляют собой компоненты, которые регистрируются и конфигурируются на этапе сборки приложения в контейнере внедрения зависимостей, а затем автоматически внедряются в конкретные страницы или компоненты, где требуется их функциональность. Пользовательские сервисы

определяются посредством C# интерфейса с префиксом "I" и соответствующего класса, реализующего данный интерфейс.

Система внедрения зависимостей включает следующие сервисы:

- **IDatabaseAccessorService** — пользовательский сервис для взаимодействия с базой данных SQLite посредством Entity Framework Core 9, обеспечивающий серверную обработку данных
- **IJiraTicketsService** — пользовательский сервис для интеграции с API Jira и Jira Service Desk, предназначенный для создания и управления заявками технической поддержки
- **ISFContactManager** — пользовательский сервис для интеграции с CRM-системой Salesforce, обеспечивающий хранение дополнительных пользовательских данных
- **IStringLocalizer** — встроенный сервис ASP.NET Core для обеспечения многоязычной поддержки приложения (русский и английский языки)
- **SignInManager** — встроенный сервис ASP.NET Core для управления процессом аутентификации пользователей
- **UserManager** — встроенный сервис ASP.NET Core для управления пользовательскими данными и авторизацией
- **AuthenticationStateProvider** — встроенный сервис ASP.NET Core для отслеживания текущего состояния аутентификации пользователя
- **NavigationManager** — встроенный сервис ASP.NET Core для программного управления навигацией между страницами приложения
- **ISnackbar** — сервис библиотеки MudBlazor для отображения всплывающих уведомлений пользователю

Маршрутизация HTTP-запросов осуществляется автоматически средствами ASP.NET Core на основе анализа Razor-страниц, при этом система самостоятельно определяет соответствие между URL-адресами и

компонентами пользовательского интерфейса, обеспечивая корректную обработку пользовательских запросов и навигацию по функциональным модулям приложения.

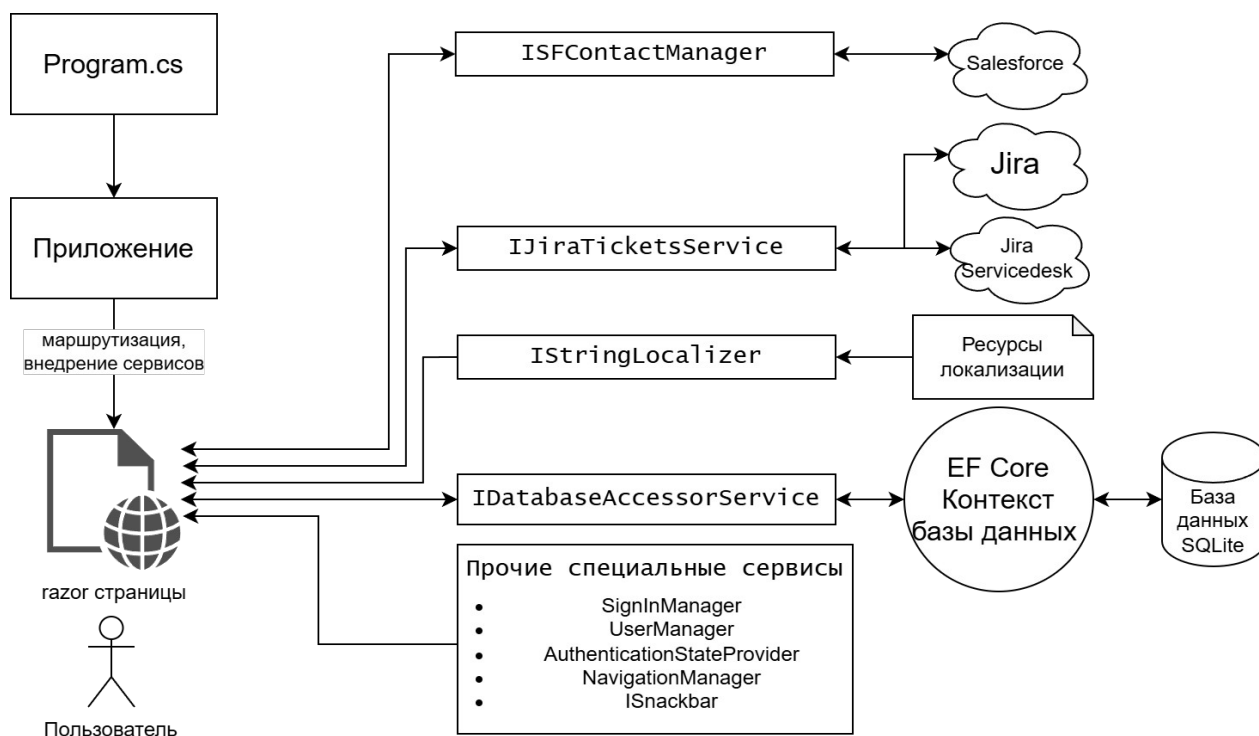


Рисунок 2.1 - Схема работы приложения на высоком уровне

2.3 Проектировка структуры базы данных

После выбора технологий необходимо приступить к проектированию структуры базы данных, что является одним из ключевых этапов разработки любого информационного приложения. Данный процесс включает в себя детальный анализ предметной области, выявление основных бизнес-сущностей, определение их атрибутов и установление логических связей между ними.

В контексте объектно-реляционного моделирования сущность представляет собой абстрактный объект реального мира, обладающий определенными характеристиками и свойствами, которые имеют значение

для разрабатываемой системы. Каждая сущность характеризуется набором атрибутов (полей), которые описывают ее свойства и хранят конкретные значения данных. Атрибуты могут быть простыми (неделимыми) или составными (состоящими из нескольких компонентов), обязательными или необязательными, а также могут служить в качестве первичных или внешних ключей для обеспечения целостности данных.

В рамках разрабатываемого приложения каждая выявленная сущность будет реализована в виде C# класса, где атрибуты сущности представлены как свойства (properties) класса с соответствующими типами данных. Entity Framework Core использует подход "Code First", при котором структура базы данных определяется на основе классов-сущностей, что обеспечивает тесную интеграцию между объектной моделью приложения и реляционной структурой базы данных.

Центральным компонентом архитектуры данных выступает класс контекста базы данных `WoasFormsDbContext`, который наследуется от `IdentityDbContext<WoasFormsAppUser>`. Данный класс служит точкой входа для всех операций с базой данных и содержит `DbSet<T>` свойства для каждой сущности, обеспечивая объектно-реляционное отображение (ORM).

Особое значение в Entity Framework Core имеют навигационные свойства (navigation properties) - специальные свойства сущностей, которые представляют связи с другими сущностями. Навигационные свойства позволяют осуществлять навигацию по объектному графу, обеспечивая прямой доступ к связанным данным без необходимости выполнения дополнительных запросов к базе данных. EF Core поддерживает различные типы связей: один-к-одному (one-to-one), один-ко-многим (one-to-many) и многие-ко-многим (many-to-many).

Навигационные свойства могут быть определены как у одной из связанных сущностей (односторонняя навигация), так и у обеих сущностей (двусторонняя навигация), что обеспечивает гибкость в проектировании модели данных и упрощает реализацию бизнес-логики приложения. EF Core способен автоматически обнаруживать и конфигурировать связи на основе соглашений, анализируя навигационные свойства и их типы.

Дополнительная конфигурация связей и ограничений может быть выполнена в методе `OnModelCreating` класса `WoasFormsDbContext` с использованием Fluent API, что позволяет точно настроить поведение Entity Framework Core в соответствии с требованиями предметной области. Данный подход обеспечивает полный контроль над генерируемой схемой базы данных.

Важным преимуществом Entity Framework Core является его способность к автоматическому созданию и обновлению структуры базы данных посредством механизма миграций (migrations). Система миграций отслеживает изменения в модели данных и генерирует соответствующие SQL-скрипты для применения этих изменений к физической базе данных, обеспечивая версионирование схемы данных и упрощая процесс развертывания приложения в различных средах.

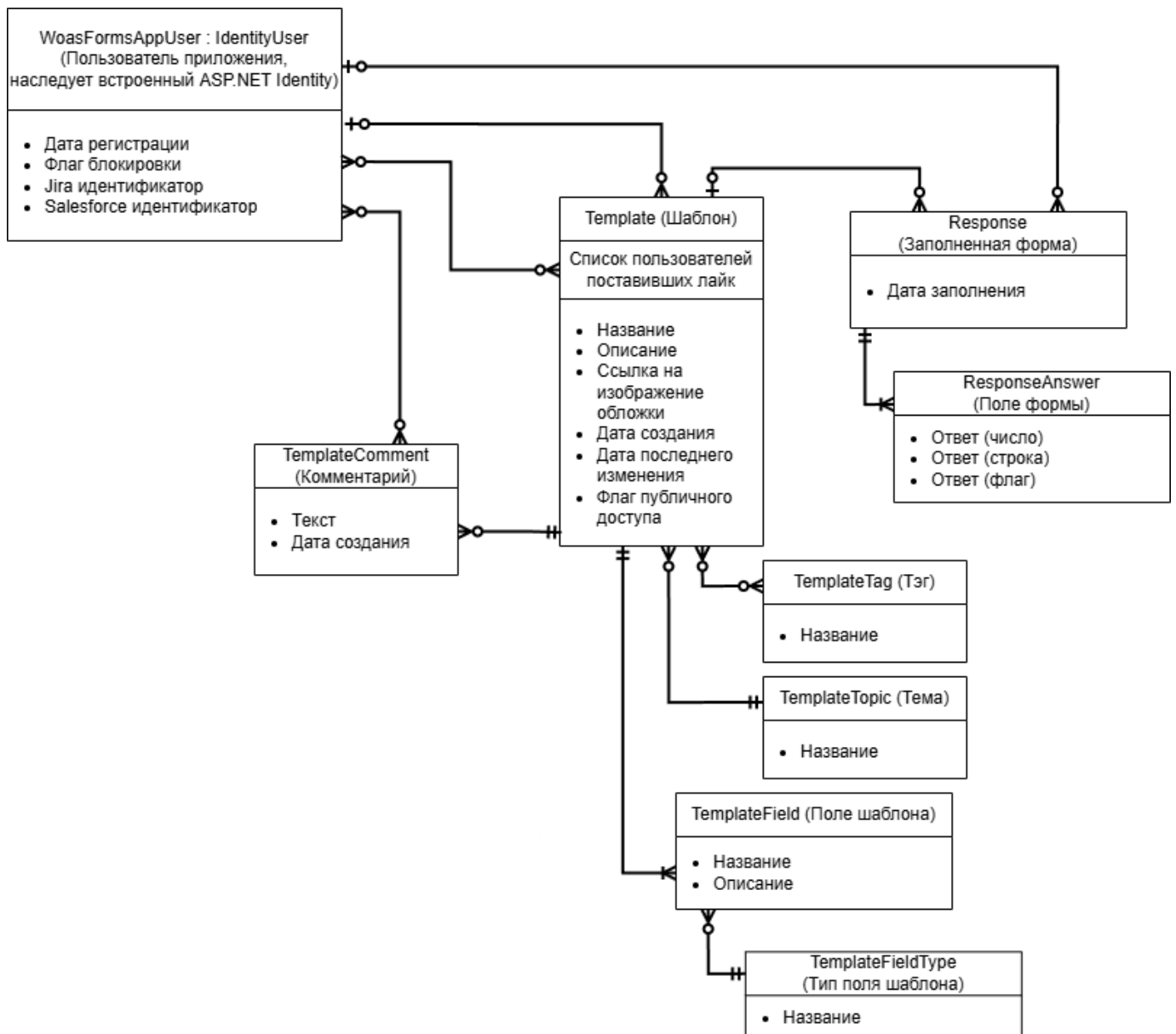


Рисунок 2.2 - Схема реляционной базы данных

Был составлен перечень сущностей, их примитивных полей и навигационных полей:

- Пользователь веб приложения (WoasFormsAppUser)
 - Дата регистрации
 - Флаг блокировки
 - Коллекция принадлежащих шаблонов (Template[])
 - Коллекция заполненных форм (Response[])

- Идентификатор аккаунта на платформе Jira
- Идентификатор аккаунта/контакта на платформе Salesforce
- Шаблон (Template)
 - Владелец (WoasFormsAppUser)
 - Дата создания
 - Дата последнего изменения
 - Название
 - Описание
 - Обложка - строка, url на изображение, отображающееся на карте шаблона
 - Тема (TemplateTopic)
 - Тэги (TemplateTag[])
 - Пользователи, поставившие лайк (WoasFormsAppUser[])
 - Комментарии (TemplateComment[])
 - Флаг публичный шаблон
 - Список форм (Response[])
 - Поля шаблона (TemplateField[])
- Поле шаблона (TemplateField)
 - Шаблон (Template)
 - Заголовок
 - Описание
 - Тип поля (TemplateFieldType)
 - Порядковая позиция
 - Флаг отображения в аналитике
- Тип поля шаблона (TemplateFieldType)
 - Название

- Заполненная форма (Response)
 - Респондент (WoasFormsAppUser)
 - Шаблон (Template)
 - Дата заполнения формы
 - Список ответов (ResponseAnswer[])
- Ответ на поле шаблона (ResponseAnswer)
 - Поле шаблона (TemplateField)
 - Ответ строка, string
 - Ответ число, int
 - Ответ флаг, bool
- Тег шаблона (TemplateTag)
 - Название
- Тема шаблона (TemplateTopic)
 - Название
- Комментарий к шаблону (TemplateComment)
 - Автор комментария (WoasFormsAppUser)
 - Шаблон (Template)
 - Текст комментария
 - Дата создания комментария

2.4 Проектировка необходимых страниц

В контексте разработки современных веб-приложений с использованием фреймворка ASP.NET Core Blazor, фундаментальным принципом архитектурного проектирования является создание четко структурированной иерархии пользовательского интерфейса, основанной на логическом разделении функциональных областей приложения.

Согласно парадигме компонентно-ориентированной разработки, каждому логически обособленному разделу приложения должна соответствовать отдельная страница, выполняющая специфические функции отображения релевантных данных и, при необходимости, обеспечивающая механизмы приема и обработки пользовательского ввода. Данный подход способствует поддержанию принципов единственной ответственности (Single Responsibility Principle) и разделения интересов (Separation of Concerns), что является краеугольным камнем качественной архитектуры программного обеспечения.

Платформа **ASP.NET Core Blazor** предоставляет разработчикам расширенный инструментарий для создания пользовательских интерфейсов, включающий в себя два основных архитектурных элемента: **razor-страницы** и **изолированные компоненты**.

Razor-страницы представляют собой полнофункциональные веб-страницы, которые служат основными точками входа для различных разделов приложения. Каждая страница инкапсулирует в себе как логику представления данных, так и механизмы взаимодействия с пользователем, обеспечивая целостный пользовательский опыт в рамках конкретной функциональной области. Особенностью архитектуры ASP.NET Core Blazor является наличие специализированной директивы **@page**, которая существенно упрощает и автоматизирует процесс настройки системы маршрутизации. Данная директива позволяет декларативно определить URL-адреса, по которым будет доступна конкретная страница, устраняя необходимость в ручной конфигурации маршрутов в отдельных файлах настроек. Применение директивы **@page** обеспечивает тесную интеграцию между логикой представления и правилами навигации, что способствует

созданию более согласованной и легко сопровождаемой архитектуры приложения.

Параллельно с созданием полноценных страниц, фреймворк предоставляет возможность разработки **изолированных компонентов** — модульных элементов пользовательского интерфейса, которые можно многократно использовать в различных контекстах приложения. Эти компоненты функционируют как переиспользуемые строительные блоки, позволяющие разработчикам создавать сложные пользовательские интерфейсы путем композиции более простых, специализированных элементов.

Применение данной архитектурной стратегии обеспечивает ряд существенных преимуществ:

- **Масштабируемость:** Модульная структура облегчает расширение функциональности приложения без нарушения существующей архитектуры.
- **Сопровождаемость:** Четкое разделение ответственности между компонентами упрощает процессы отладки, тестирования и модификации кода.
- **Переиспользование:** Компонентный подход способствует созданию библиотек переиспользуемых элементов интерфейса, что повышает эффективность разработки.
- **Тестируемость:** Изолированные компоненты легче поддаются модульному тестированию, что способствует повышению качества программного продукта.

Таким образом, архитектурная модель ASP.NET Core Blazor, основанная на сочетании страничной структуры и компонентного подхода,

представляет собой эффективное решение для создания современных, масштабируемых веб-приложений с высоким уровнем сопровождаемости и переиспользования кода.

Для приложения определён следующий набор страниц, требующих реализации:

- Главная страница;
- Регистрация нового аккаунта;
- Вход в аккаунт;
- Личный кабинет;
- Шаблон — информация о шаблоне (Название, описание), ссылка на заполнение, ссылка для редактирования, сбора и анализа информации, (доступ только для владельца шаблона и администратора) элементы социального взаимодействия (лайки, комментарии)
- Редактор шаблона; (как для создания, так и редактирования)
- Заполнение формы по шаблону;
- Просмотр заполненной формы шаблона;
- Полученные формы — страница с данными форм пользователей, доступна только для владельца шаблона и администраторов;
- Поиск шаблонов — страница с полем текстового поиска, интерактивными фильтрами и отображаемым результатом поиска;
- Панель администрирования;
- Отправка заявления техподдержки;

2.5 Проектировка необходимых ASP.NET Core сервисов

В качестве структурного подхода был выбран микс Сервисов и дополнительного razor кода для некоторых индивидуальных страниц. Сервисы будут отвечать за большую часть функционала приложения, в том числе за обращение к базе данных через EF Core сущности, (IDatabaseAccessorService) сервис обращения к платформе Jira (IJiraTicketsService) и сервис обращения к платформе Salesforce Cloud (ISFContactManager). Для описание требуемых функций используются C# интерфейсы, данные интерфейсы будут в дальнейшем конкретно реализованы в качестве классов. Это повторяет методологию ASP.NET Core, где в приложении регистрируются сервис классы, как реализации абстрактных интерфейсов. Это помогает разделить требования проекта от конкретных реализаций, например, для создания альтернативных сервис классов.

Было составлено описание следующих интерфейсов:

IDatabaseAccessorService:

- Task<HashSet<TemplateTopic>> GetTopics();
- Task<HashSet<TemplateTag>> GetTags();
- Task<HashSet<TemplateFieldType>> GetTemplateFieldTypes();
- Dictionary<TemplateOrderMode, TemplateOrderModeData>
GetTemplateOrdersData();
- Task<IList<Template>> SearchTemplates(TemplateOrderMode, string?
query, string? username, string? tag, string? topic);
- Task<bool> GetCurrentUserOwnsTemplate(int templateId);
- Task<Template?> GetTemplate(int templateId);

- Task<IEnumerable<Template>> GetAvailableTemplates();
- Task<IEnumerable<Template>> GetTemplatesByOwner(string userName);
- Task<IEnumerable<Template>> GetTemplatesByCurrentUser();
- Task<Template?> CreateTemplate(Template template);
- Task<Template?> UpdateTemplate(Template newTemplate, int? templateId);
- Task DeleteTemplate(int templateId);
- Task LikeTemplate(int templateId, bool like);
- Task CommentOnTemplate(int templateId, string commentText);
- Task<bool> GetCurrentUserOwnsResponse(int responseId);
- Task<Response?> CreateResponse(Response response);
- Task<Response?> GetResponse(int responseId);
- Task<IEnumerable<Response>> GetResponsesByTemplate(int templateId);
- Task<IEnumerable<Response>> GetResponsesByRespondent(string userId);
- Task<IEnumerable<Response>> GetResponsesByCurrentUser();
- Task<Dictionary<TemplateTag, int>> GetTemplateCountsByTag();
- Task<WoasFormsAppUser?> GetUser(string userId);
- Task<List<WoasFormsAppUser>> GetAllUsers();
- Task<WoasFormsAppUser?> GetCurrentUser();

- Task DeleteUser(string userId);
- Task GiveUserRole(string userId, string roleName);
- Task RevokeUserRole(string userId, string roleName);
- Task BlockUser(string userId);
- Task UnblockUser(string userId);
- Task<List<string>> GetUserRoles(string userId);
- IJiraTicketsService
- Task<JiraCustomerView?> CreateCustomerFromCurrentUser(string? overrideEmail);
- Task<JiraCustomerView?> GetCustomerFromCurrentUser();
- Task<JiraTicketView?> CreateTicketByCurrentUser(NewTicketModel model);
- Task<JiraTicketView?> GetTicketById(string ticketId);
- ISFContactManager
- Task<sfUserDataView?> CreateCurrentUserSFContact();
- Task<sfUserDataView?> GetCurrentUserSFContact();
- Task<bool> UpdateCurrentUserSFContact(sfUserDataView sfUserDataView);

Глава 3. Реализация приложения

3.1 Реализация сущностей приложения. Миграция базы данных.

В рамках реализации слоя доступа к данным был создан класс *WoasFormsDbContext*, который наследуется от встроенного *IdentityDbContext<WoasFormsAppUser>*. Данный подход позволяет интегрировать систему управления пользователями ASP.NET Core Identity с пользовательской моделью данных приложения. Внутри контекста в методе *OnModelCreating()* выполняется явная настройка отношений между сущностями с использованием Fluent API, что обеспечивает точное определение типов связей (один-к-одному, один-ко-многим, многие-ко-многим) и дополнительных ограничений на уровне базы данных.

Классы сущностей представляют собой гибридные модели, содержащие как примитивные свойства (строки, числа, даты), так и навигационные свойства для связанных объектов. Навигационные свойства играют ключевую роль в процессе создания миграций Entity Framework Core, автоматически определяя структуру внешних ключей и индексов. В дальнейшем эти свойства значительно упрощают разработку сервисов приложения, предоставляя fluent-интерфейс для работы с данными через LINQ-запросы, что делает работу с базой данных намного удобнее по сравнению с прямым обращением к БД через SQL-запросы.

3.2 Реализация сервисов приложения

В архитектуре приложения реализованы три основных сервиса, которые выполняют роль слоев абстракции между пользовательским интерфейсом и внешними системами. *DatabaseAccessorService* служит промежуточным слоем между Razor-страницами и Entity Framework Core, инкапсулируя логику работы с базой данных и предоставляя упрощенный интерфейс для выполнения CRUD-операций. *JiraTicketsService* обеспечивает

взаимодействие с REST API Jira для управления тикетами техподдержки, скрывая сложность HTTP-запросов и сериализации данных за простыми методами. *SFContactManager* аналогично реализует слой абстракции для работы с REST API Salesforce CRM, предоставляя удобные методы для управления контактами.

Все три сервиса регистрируются в контейнере внедрения зависимостей и внедряются только в те Razor-страницы, где их функциональность действительно необходима. Такой подход соответствует принципу разделения ответственности и позволяет избежать избыточных зависимостей, делая код более поддерживаемым и тестируемым. Каждая страница получает доступ только к тем сервисам, которые требуются для реализации её конкретной логики.

3.3 Реализация razor страниц приложения

Пользовательский интерфейс приложения реализован с использованием Razor-страниц, исходный код которых представляет собой смесь HTML-разметки и минимального количества C#-кода. HTML-разметка отвечает за визуальный дизайн страницы с использованием компонентов MudBlazor, обеспечивая современный и отзывчивый интерфейс. C#-код выполняет роль связующего звена, вызывая соответствующие функции внедренных сервисов для получения и обработки данных.

Такая архитектура обеспечивает четкое разделение презентационной логики и бизнес-логики, при этом сама страница остается максимально простой и фокусируется на отображении данных и обработке пользовательского ввода. Со всеми деталями реализации каждой страницы можно ознакомиться по ссылке на GitHub репозиторий проекта.

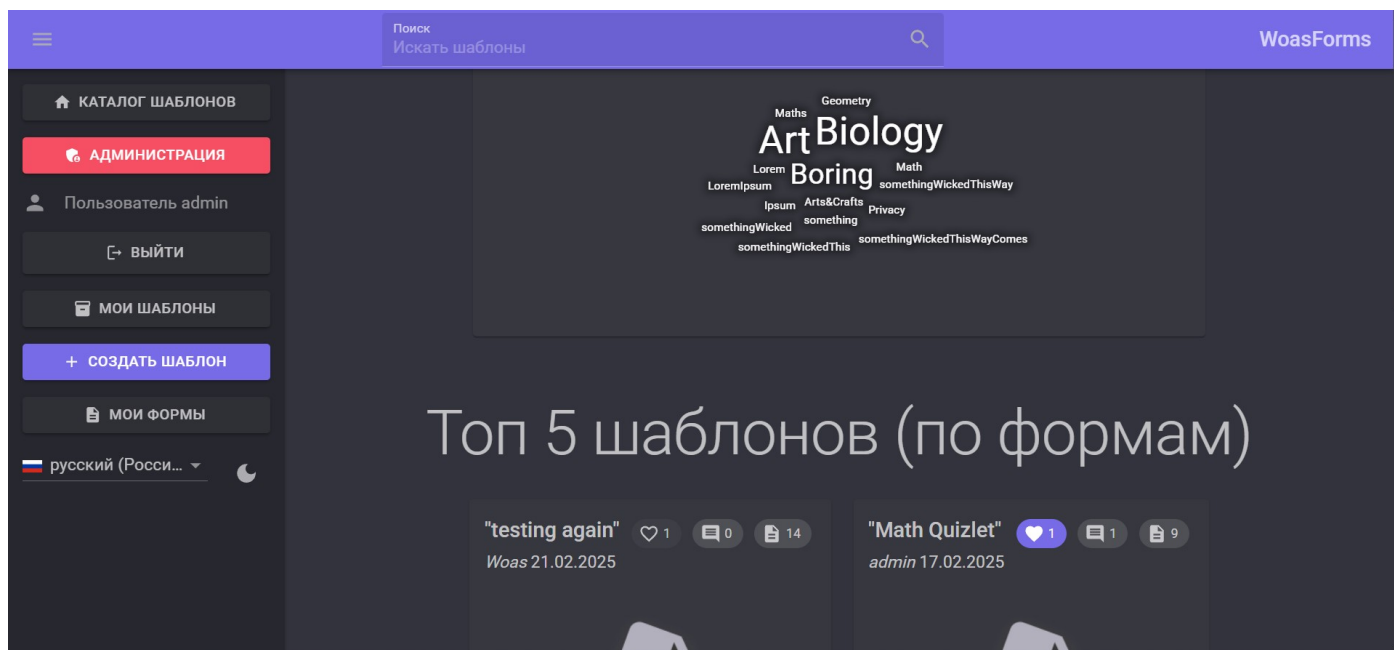


Рисунок 3.1: Дизайн главной страницы приложения

На данный момент система анализа данных из собранных форм достаточно простая, и не требует разработки отдельного сервиса. Все функции находятся в исходном коде соответствующей razor-страницы.

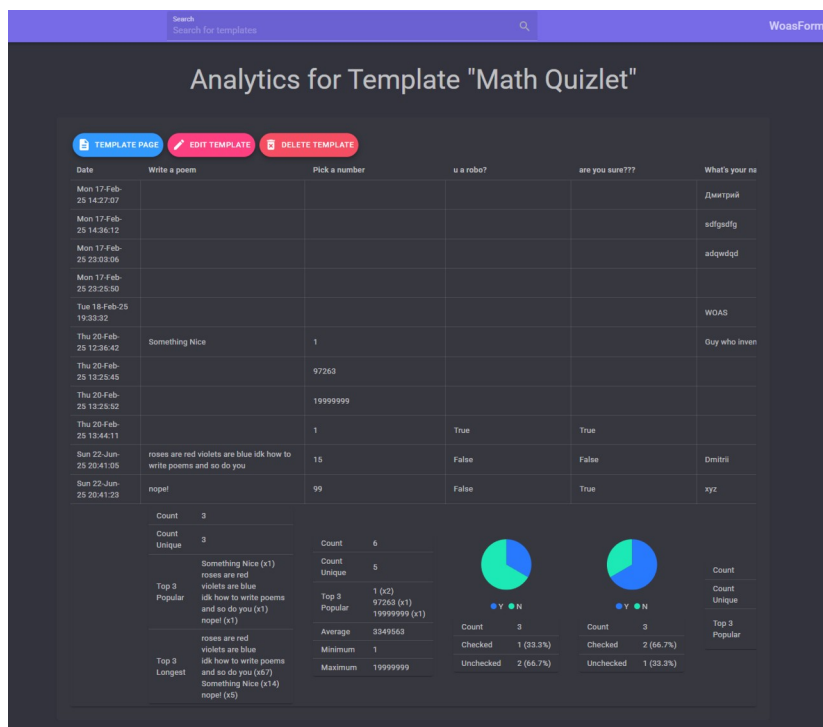


Рисунок 3.2: Страница аналитики собранных форм

3.4 Реализация сервисов REST API, настройка API доступа

Application Programming Interface (API) представляет собой набор протоколов и инструментов для взаимодействия между различными программными компонентами. В контексте веб-разработки REST API обеспечивает стандартизированный способ обмена данными между приложениями через HTTP-протокол. Для интеграции с внешними платформами в приложении используются встроенные в ASP.NET Core HTTP-клиенты, которые позволяют выполнять запросы к удаленным сервисам.

Сервисы для интеграции с внешними платформами реализованы с использованием различных методов аутентификации в зависимости от требований каждой системы. *JiraTicketsService* использует базовую аутентификацию с логином и паролем для доступа к REST API Jira, что позволяет выполнять операции создания, чтения и обновления тикетов техподдержки. *SFContactManager* реализует более сложную схему аутентификации через OAuth2 токены для работы с Salesforce CRM, включая сервис управления токенами для автоматического обновления истекших токенов доступа.

Все API-ключи и учетные данные для доступа к внешним сервисам внедряются из системных файлов конфигурации и никогда не сохраняются в репозитории исходного кода. Конфиденциальная информация хранится исключительно на машине-сервере в защищенных конфигурационных файлах, что обеспечивает безопасность доступа к внешним системам и соответствует лучшим практикам разработки.

3.5 Доставка приложения на облачный сервис, конфигурация облачной машины. Настройка доменного имени

Для обеспечения публичного доступа к разработанному веб-приложению было выполнено его развертывание в облачной инфраструктуре Oracle Cloud Infrastructure (OCI). В рамках данного этапа была создана виртуальная машина на базе операционной системы Linux Ubuntu, которая предоставила необходимые вычислительные ресурсы для функционирования приложения в продакшн-среде.

Процесс развертывания включал несколько ключевых этапов. Первоначально приложение было скомпилировано в исполняемый формат для Linux-платформы с использованием команды `dotnet publish` с соответствующими параметрами кроссплатформенной компиляции. Полученные исполняемые файлы, а также копия базы данных SQLite были переданы на удаленный сервер посредством SSH-протокола, обеспечивающего безопасную передачу данных.

На виртуальной машине был установлен и настроен веб-сервер Nginx, выполняющий функции обратного прокси-сервера для перенаправления HTTP-запросов к ASP.NET Core приложению. Конфигурация Nginx включала настройку проксирования запросов на локальный порт, на котором функционирует Blazor-приложение. Для обеспечения автоматического запуска и управления жизненным циклом приложения был создан системный сервис Linux (systemd service), который автоматически запускает приложение при старте системы и перезапускает его в случае сбоя, передавая необходимые параметры конфигурации и переменные окружения.

Все операции по конфигурированию сервера, установке зависимостей, настройке сервисов и управлению файлами выполнялись через SSH-консоль,

что обеспечило безопасный удаленный доступ к серверу и возможность администрирования без графического интерфейса.

Завершающим этапом развертывания стала регистрация доменного имени через сервис Namecheap по специальной акционной цене \$1 на годовой период. Доменное имя было привязано к IP-адресу виртуальной машины через настройку DNS-записей типа A, что позволило обеспечить доступ к веб-приложению по понятному пользователям URL-адресу. Интеграция доменного имени с Nginx потребовала соответствующих изменений в конфигурационных файлах веб-сервера для корректной обработки запросов к указанному домену.

В результате выполненных работ веб-приложение стало доступно для пользователей через сеть Интернет с любых устройств - персональных компьютеров, планшетов и мобильных телефонов, что подтвердило успешность процесса развертывания и готовность системы к эксплуатации в реальных условиях.

ЗАКЛЮЧЕНИЕ

В рамках выполнения данного исследования были достигнуты все поставленные цели и решены основные задачи разработки. Результатом проведенной работы стало создание полнофункционального веб-приложения для управления динамическими формами с возможностью создания, редактирования и публикации шаблонов, а также сбора и анализа данных из заполненных пользователями форм.

В ходе исследования был проведен анализ существующих программных решений аналогичного функционала, что позволило выявить основные требования к разрабатываемому приложению и обосновать выбор технологического стека. Были изучены современные технологии проектирования веб-приложений, включая принципы создания интерактивных пользовательских интерфейсов и методы организации архитектуры приложений.

В процессе проектирования и реализации была создана архитектура приложения и модель данных. Реализация была выполнена с использованием фреймворка ASP.NET Core, Entity Framework Core 9 для работы с базой данных SQLite и библиотеки MudBlazor для создания пользовательского интерфейса.

Созданное решение демонстрирует практическую применимость изученных технологий и обладает интуитивно понятным интерфейсом, высокой производительностью и возможностями для дальнейшего развития. Все задачи исследования были успешно выполнены, поставленные цели достигнуты.

Были установлены планы по дальнейшему развитию проекта:

- Полноценная система регистрации — на данный момент нет системы подтверждения владельца почты, так же у пользователя нет возможности самостоятельно удалить аккаунт без связи с техподдержкой/администрацией;
- Более углубленный анализ данных форм. Возможно с утилизацией анализа нейронными сетями, которым наряду с результатами форм будут предоставляться мета данные шаблонов; (лайки, комментарии, просмотры, рейтинги респондентов и создателя и т.д.)
- Полнотекстовый поиск по шаблонам. SQLite поддерживает технику эффективного полнотекстового поиска на уровне базы данных, с помощью расширения FTS5. Планируется изучение и утилизация этого расширения;
- Поддержка загрузки пользовательских изображений в качестве обложек шаблонов, возможно используя сторонние сервисы облачного хранения данных;

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

Электронные ресурсы

1. Google Forms. [Электронный ресурс]. – Режим доступа: <https://forms.google.com> – Дата обращения: 15.01.2025
2. Microsoft Forms. [Электронный ресурс]. – Режим доступа: <https://forms.microsoft.com> – Дата обращения: 08.01.2025
3. Typeform. [Электронный ресурс]. – Режим доступа: <https://typeform.com> – Дата обращения: 22.01.2025
4. JotForm. [Электронный ресурс]. – Режим доступа: <https://jotform.com> – Дата обращения: 11.01.2025
5. SurveyMonkey. [Электронный ресурс]. – Режим доступа: <https://surveymonkey.com> – Дата обращения: 27.01.2025
6. Reddit. [Электронный ресурс]. – Режим доступа: <https://reddit.com> – Дата обращения: 19.01.2025
7. ArtStation. [Электронный ресурс]. – Режим доступа: <https://artstation.com> – Дата обращения: 05.01.2025
8. Dribbble. [Электронный ресурс]. – Режим доступа: <https://dribbble.com> – Дата обращения: 31.01.2025
9. React. [Электронный ресурс]. – Режим доступа: <https://react.dev/> – Дата обращения: 15.01.2025
10. Vue.js. [Электронный ресурс]. – Режим доступа: <https://vuejs.org/> – Дата обращения: 22.01.2025

- 11.ASP.NET Core Blazor. [Электронный ресурс]. – Режим доступа: <https://dotnet.microsoft.com/en-us/apps/aspnet/web-apps/blazor> – Дата обращения: 08.01.2025
- 12.Node.js. [Электронный ресурс]. – Режим доступа: <https://nodejs.org/> – Дата обращения: 18.01.2025
- 13.Express.js. [Электронный ресурс]. – Режим доступа: <https://expressjs.com/> – Дата обращения: 11.01.2025
- 14.Django. [Электронный ресурс]. – Режим доступа: <https://www.djangoproject.com/> – Дата обращения: 27.01.2025
- 15.ASP.NET Core. [Электронный ресурс]. – Режим доступа: <https://dotnet.microsoft.com/en-us/apps/aspnet> – Дата обращения: 03.01.2025
- 16.PostgreSQL. [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/> – Дата обращения: 20.01.2025
- 17.MySQL. [Электронный ресурс]. – Режим доступа: <https://www.mysql.com/> – Дата обращения: 14.01.2025
- 18.SQLite. [Электронный ресурс]. – Режим доступа: <https://www.sqlite.org/> – Дата обращения: 25.01.2025
- 19.Material-UI. [Электронный ресурс]. – Режим доступа: <https://mui.com/> – Дата обращения: 07.01.2025
- 20.Bootstrap. [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/> – Дата обращения: 16.01.2025
- 21.MudBlazor. [Электронный ресурс]. – Режим доступа: <https://mudblazor.com/> – Дата обращения: 12.01.2025

22. Microsoft Azure. [Электронный ресурс]. – Режим доступа: <https://azure.microsoft.com/> – Дата обращения: 19.01.2025
23. Amazon Web Services. [Электронный ресурс]. – Режим доступа: <https://aws.amazon.com/> – Дата обращения: 05.01.2025
24. Oracle Cloud Infrastructure. [Электронный ресурс]. – Режим доступа: <https://www.oracle.com/cloud/> – Дата обращения: 26.01.2025
25. Jira REST API. [Электронный ресурс]. – Режим доступа: <https://developer.atlassian.com/server/jira/platform/rest-apis/> – Дата обращения: 04.02.2025
26. Salesforce REST API. [Электронный ресурс]. – Режим доступа: https://developer.salesforce.com/docs/atlas.en-us.api_rest.meta/api_rest/ – Дата обращения: 05.02.2025

Книги

1. Connolly, T., & Begg, C. Database systems: a practical approach to design, implementation, and management. – Pearson, 2015.

Интернет-ресурсы

1. Коротких Дмитрий Андреевич, WoasFormsApp source code. // Github – 2025. [Электронный ресурс]. – Режим доступа: <https://github.com/WeaselOnaStick/WoasFormsApp> – Дата обращения: 10.03.2025.
2. Коротких Дмитрий Андреевич, Woas Forms App. [Электронный ресурс]. – Режим доступа: <http://woasforms.site/> - Дата обращения: 10.03.2025

Реализация сервиса классов сущностей и класса контекста базы данных

```

public class WoasFormsDbContext(DbContextOptions<WoasFormsDbContext> options) :
IdentityDbContext<WoasFormsAppUser>(options)
{
    public DbSet<Template> Templates { get; set; }
    public DbSet<TemplateTag> TemplateTags { get; set; }
    public DbSet<TemplateTopic> TemplateTopics { get; set; }
    public DbSet<TemplateField> TemplateFields { get; set; }
    public DbSet<TemplateFieldType> FieldTypes { get; set; }
    public DbSet<Response> Responses { get; set; }
    public DbSet<TemplateComment> TemplateComments { get; set; }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        builder.Entity<WoasFormsAppUser>().HasMany(u => u.OwnedTemplates).WithOne(t => t.Owner);
        builder.Entity<WoasFormsAppUser>().HasMany(u => u.Responses).WithOne(r => r.Respondent);
        builder.Entity<WoasFormsAppUser>().HasMany<TemplateComment>().WithOne(c => c.User);
        builder.Entity<Template>().HasMany(t => t.Tags).WithMany();
        builder.Entity<Template>().HasMany(t => t.UsersWhoLiked).WithMany();
        builder.Entity<Template>().HasMany(t => t.Comments).WithOne(c => c.Template);
        builder.Entity<Template>().HasMany(t => t.AllowedUsers).WithMany();
        builder.Entity<Template>().HasMany(t => t.Responses).WithOne(r => r.Template);
        builder.Entity<Template>().HasMany(t => t.Fields).WithOne(f => f.Template);
        base.OnModelCreating(builder);
    }
}

public class WoasFormsAppUser : IdentityUser
{
    public DateTime? RegisteredAt { get; set; }
    public bool IsBlocked { get; set; } = false;
    public ICollection<Response> Responses { get; set; }
    public ICollection<Template> OwnedTemplates { get; set; }
    public string SalesforceAccountId { get; set; } = "";
    public string SalesforceContactId { get; set; } = "";
    public string JiraAccountId { get; set; } = "";
}

public class TemplateTopic
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public string Title { get; set; } = "";
}

public class TemplateTag
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public required string Title { get; set; }
}

public class TemplateField
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public Template Template { get; set; }
    public bool Hidden { get; set; } = false;
    public string Title { get; set; } = "";
    public string Description { get; set; } = "";
    public TemplateFieldType Type { get; set; }
    public int Position { get; set; }

    public bool ShowInAnalytics { get; set; } = true;
}

```

```

public class TemplateFieldType
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public string Name { get; set; } = "";
}
public class TemplateComment
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    public WoasFormsAppUser User { get; set; }
    public Template? Template { get; set; }
    public string Text { get; set; } = "";
    public DateTime PostedAt { get; set; } = DateTime.UtcNow;
}
public class Template
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public WoasFormsAppUser? Owner { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime LastModifiedAt { get; set; } = DateTime.UtcNow;
    public string Title { get; set; } = "";
    public string? Description { get; set; } = "";
    public string? CoverImageUrl { get; set; } = "";
    public int TopicId { get; set; } = 0;
    public TemplateTopic Topic { get; set; }
    public ICollection<TemplateTag> Tags { get; set; } = new HashSet<TemplateTag>();
    public ICollection<WoasFormsAppUser> UsersWhoLiked { get; set; } = new HashSet<WoasFormsAppUser>();
    public ICollection<TemplateComment> Comments { get; set; } = new HashSet<TemplateComment>();
    public bool Public { get; set; } = true;
    public ICollection<WoasFormsAppUser> AllowedUsers { get; set; } = new HashSet<WoasFormsAppUser>();
    public ICollection<Response> Responses { get; set; } = new HashSet<Response>();
    public required List<TemplateField> Fields { get; set; } = new List<TemplateField>();
}
public class Response
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public WoasFormsAppUser? Respondent { get; set; }
    public DateTime CreationDate { get; set; } = DateTime.UtcNow;
    public Template? Template { get; set; }
    public List<ResponseAnswer> Answers { get; set; } = new List<ResponseAnswer>();
}
public class ResponseAnswer
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    public TemplateField Field { get; set; }
    public string? AnswerSingleLine { get; set; }
    public string? AnswerMultiLine { get; set; }
    public int? AnswerPositiveInt { get; set; }
    public bool? AnswerCheckBox { get; set; }
}

```

Реализация сервиса IDatabaseAccessorService

```

namespace WoasFormsApp.Services
{
    public class DatabaseAccessorService : IDatabaseAccessorService
    {
        const int MAX_AMOUNT_OF_FIELDS_PER_TYPE = 4;
        WoasFormsDbContext _ctx;
        AuthenticationStateProvider _asp;
        UserManager<WoasFormsAppUser> _users;
        SignInManager<WoasFormsAppUser> _sign;
        public DatabaseAccessorService(
            WoasFormsDbContext context,
            AuthenticationStateProvider authenticationStateProvider,
            UserManager<WoasFormsAppUser> userManager,
            SignInManager<WoasFormsAppUser> signInManager)
        {
            _ctx = context;
            _asp = authenticationStateProvider;
            _users = userManager;
            _sign = signInManager;
        }
        private WoasFormsAppUser? _curUserCache = null;
        private bool _curUserIsCached = false;
        private bool _curUserHasAdminCache;
        private bool _curUserHasAdminIsCached = false;
        public async Task<WoasFormsAppUser?> GetCurrentUser()
        {
            if (!_curUserIsCached)
            {
                var user = (await _asp.GetAuthenticationStateAsync()).User;
                string userName = user.Identity.Name;
                if (userName == null) return null;
                var resUser = await _users.FindByNameAsync(userName);
                _curUserCache = resUser;
                _curUserIsCached = true;
            }
            return _curUserCache;
        }
        private async Task<bool> CurrentUserHasAdmin()
        {
            if (!_curUserHasAdminIsCached)
            {
                var user = await GetCurrentUser();
                if (user == null) return false;
                _curUserHasAdminCache = await _users.IsInRoleAsync(user, "Admin");
                _curUserHasAdminIsCached = true;
            }
            return _curUserHasAdminCache;
        }
        private async Task<bool> CurrentUserHasPowerOverTarget(WoasFormsAppUser targetUser)
            => (await CurrentUserHasAdmin()) || (await GetCurrentUser() == targetUser);
        private async Task<bool> CurrentUserHasPowerOverTarget(string userId)
            => await CurrentUserHasPowerOverTarget(await GetUserId(userId));

        private async Task<WoasFormsAppUser> GetUserId(string userId) => await _users.FindByIdAsync(userId);
        private async Task<WoasFormsAppUser> GetUserName(string userName) => await _users.FindByNameAsync(userName);
        public async Task<HashSet<TemplateTopic>> GetTopics() =>
            await _ctx.TemplateTopics.ToHashSetAsync();
        public async Task<HashSet<TemplateTag>> GetTags() =>
            await _ctx.TemplateTags.ToHashSetAsync();
        public async Task<HashSet<TemplateFieldType>> GetTemplateFieldTypes() =>
            await _ctx.FieldTypes.ToHashSetAsync();
        public bool ValidateTemplate(Template template)
        {
            if (string.IsNullOrEmpty(template.Title)) return false;
            if (template.Fields.Count == 0) return false;
            if (template.Fields.GroupBy(f => f.Type.Id).Select(g => g.Count()).Max() > MAX_AMOUNT_OF_FIELDS_PER_TYPE) return false;
            return true;
        }
        private async Task<HashSet<TemplateTag>> CreateNecessaryTags(Template submission)
        {
            HashSet<TemplateTag> syncedTagList = new HashSet<TemplateTag>();
            HashSet<TemplateTag> newTags = new HashSet<TemplateTag>();
            foreach (var tagName in submission.Tags.Select(t => t.Title))
            {
                TemplateTag? foundTag = await _ctx.TemplateTags.FirstOrDefaultAsync(t => t.Title == tagName);
            }
        }
    }
}

```

```

        if (foundTag != null)
            syncedTagList.Add(foundTag);
        else
            newTags.Add(new TemplateTag { Title = tagName });
    }
    if (newTags.Any())
    {
        await _ctx.TemplateTags.AddRangeAsync(newTags);
        await _ctx.SaveChangesAsync();
        syncedTagList.UnionWith(newTags);
    }
    return syncedTagList;
}
public async Task<Template?> CreateTemplate(Template template)
{
    template.Owner ??= await GetCurrentUser();
    template.Fields = template.Fields.Select(f =>
        new TemplateField
        {
            Title = f.Title,
            Template = template,
            Description = f.Description,
            Hidden = false,
            ShowInAnalytics = f.ShowInAnalytics,
            Type = f.Type,
            Position = f.Position,
        }
    ).ToList();
    template.Tags = await CreateNecessaryTags(template);
    template.CreatedAt = DateTime.UtcNow;
    template.LastModifiedAt = template.CreatedAt;
    if (!ValidateTemplate(template)) return null;
    var freshTemplate = await _ctx.Templates.AddAsync(template);
    try
    {
        await _ctx.SaveChangesAsync();
    }
    catch (Exception)
    {
        return null;
    }
    return freshTemplate.Entity;
}
private static IQueryable<Template> GetTemplateDetails(IQueryable<Template> query)
{
    return query
        .Include(t => t.Owner)
        .Include(t => t.Responses)
        .Include(t => t.Fields).ThenInclude(f => f.Type)
        .Include(t => t.Topic)
        .Include(t => t.Tags)
        .Include(t => t.Responses)
        .Include(t => t.UsersWhoLiked)
        .Include(t => t.Comments)
        .Include(t => t.AllowedUsers);
}
private async Task<IQueryable<Template>> TemplateAuthVisibility(IQueryable<Template> query)
{
    var curUserHasAdmin = await CurrentUserHasAdmin();
    if (curUserHasAdmin) return query;
    var curUser = await GetCurrentUser();
    return query.Where(t => t.Public || t.Owner != null && (t.Owner == curUser || curUser != null && t.AllowedUsers.Contains(curUser)));
}
private async Task<bool> TemplateAuthManage(Template template)
{
    var curUserHasAdmin = await CurrentUserHasAdmin();
    if (curUserHasAdmin) return true;
    if (template.Owner == null) return false;
    var curUser = await GetCurrentUser();
    return template.Owner == curUser;
}
public static Dictionary<TemplateOrderMode, TemplateOrderModeData> TemplateOrdersData = new Dictionary<TemplateOrderMode,
TemplateOrderModeData>
{
    {TemplateOrderMode.Newest, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_CREATED_D", Direction =
SortDirection.Descending, Selector = x => x.CreatedAt, Icon = Icons.Material.Filled.History } },
    {TemplateOrderMode.Oldest, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_CREATED_A", Direction =
SortDirection.Ascending, Selector = x => x.CreatedAt, Icon = Icons.Material.Filled.History } },
    {TemplateOrderMode.MostLiked, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_LIKES_D", Direction =
SortDirection.Descending, Selector = x => x.UsersWhoLiked.Count, Icon = Icons.Material.Filled.Favorite } },

```

```

        {TemplateOrderMode.LeastLiked, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_LIKES_A", Direction =
SortDirection.Ascending, Selector = x => x.UsersWhoLiked.Count, Icon = Icons.Material.Filled.HeartBroken } },
        {TemplateOrderMode.MostCommented, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_COMMENTS_D", Direction =
SortDirection.Descending, Selector = x => x.Comments.Count, Icon = Icons.Material.Filled.Comment } },
        {TemplateOrderMode.LeastCommented, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_COMMENTS_A", Direction =
SortDirection.Ascending, Selector = x => x.Comments.Count, Icon = Icons.Material.Filled.Comment } },
        {TemplateOrderMode.MostResponded, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_FORMS_D", Direction =
SortDirection.Descending, Selector = x => x.Responses.Count, Icon = Icons.Material.Filled.Description } },
        {TemplateOrderMode.LeastResponded, new TemplateOrderModeData{ DisplayName="SEARCH_TEMPLATE_FORMS_A", Direction =
SortDirection.Ascending, Selector = x => x.Responses.Count, Icon = Icons.Material.Filled.Description } },
    };
    public Dictionary<TemplateOrderMode, TemplateOrderModeData> GetTemplateOrdersData() => TemplateOrdersData;
    public async Task<IList<Template>> SearchTemplates(TemplateOrderMode order = TemplateOrderMode.Newest, string? query = default, string?
username = default, string? tag = default, string? topic = default)
    {
        var res = (await GetAvailableTemplates()).ToList();
        if (!string.IsNullOrEmpty(query))
            res = await _ctx.Templates.Where(t =>
                (!string.IsNullOrEmpty(t.Title) && (t.Title.Contains(query) || query.Contains(t.Title))) ||
                (!string.IsNullOrEmpty(t.Description) && (t.Description.Contains(query) || query.Contains(t.Description))) ||
                t.Fields.Any(f => (!string.IsNullOrEmpty(f.Title) && (f.Title.Contains(query) || query.Contains(f.Title)))) ||
                t.Fields.Any(f => (!string.IsNullOrEmpty(f.Description) && (f.Description.Contains(query) || query.Contains(f.Description))))
            ).ToListAsync();

        if (!string.IsNullOrEmpty(username))
            res = res.Where(t => (t.Owner != null && t.Owner.UserName != null && t.Owner.UserName.Contains(username))).ToList();
        if (!string.IsNullOrEmpty(tag))
            res = res.Where(template => (template.Tags.Any(t =>
                t.Title.Contains(tag) || tag.Contains(t.Title)
            ))).ToList();
        if (!string.IsNullOrEmpty(topic))
            res = res.Where(template => template.Topic.Title == topic).ToList();
        var curOrderModeData = TemplateOrdersData[order];
        return res.OrderByDirection(curOrderModeData.Direction, curOrderModeData.Selector).ToList();
    }
    public async Task<bool> GetCurrentUserOwnsTemplate(int templateId)
    {
        var template = await GetTemplate(templateId);
        return await TemplateAuthManage(template);
    }
    public async Task<Template?> GetTemplate(int templateId)
    {
        var res = _ctx.Templates.Where(t => t.Id == templateId);
        res = await TemplateAuthVisibility(res);
        res = GetTemplateDetails(res);
        return await res.FirstOrDefaultAsync();
    }
    public async Task<IEnumerable<Template>> GetAvailableTemplates()
    {
        var res = _ctx.Templates.AsQueryable();
        res = await TemplateAuthVisibility(res);
        res = GetTemplateDetails(res);
        return res;
    }
    public async Task<IEnumerable<Template>> GetTemplatesByOwner(string userName)
    {
        var res = _ctx.Templates.Where(t => t.Owner.UserName == userName);
        res = await TemplateAuthVisibility(res);
        res = GetTemplateDetails(res);
        return res;
    }
    public async Task<IEnumerable<Template>> GetTemplatesByCurrentUser()
    {
        var curUser = await GetCurrentUser();
        var res = _ctx.Templates.Where(t => t.Owner == curUser);
        //res = await TemplateAuthVisibility(res);
        res = GetTemplateDetails(res);
        return res;
    }
    public async Task<Template?> UpdateTemplate(Template submittedTemplate, int? templateId = null)
    {
        if (!ValidateTemplate(submittedTemplate)) return null;
        templateId ??= submittedTemplate.Id;
        Template? obsoleteTemplate = await _ctx.Templates.FindAsync(templateId);
        if (obsoleteTemplate == null) return null;
        if (!await TemplateAuthManage(obsoleteTemplate)) return null;
        obsoleteTemplate.LastModifiedAt = DateTime.UtcNow;
        obsoleteTemplate.Tags = await CreateNecessaryTags(submittedTemplate);
        obsoleteTemplate.Fields.Sort((a, b) => a.Position.CompareTo(b.Position));
        submittedTemplate.Fields.Sort((a, b) => a.Position.CompareTo(b.Position));
        // <TemplateField.Id, TypeRelativeIndex> (Type Relative Index - how many fields of this type precede this field)
    }

```

```

static Dictionary<int, int> CountFieldsByType(Template t)
=> t.Fields.GroupBy(f => f.Type.Id)
.SelectMany(g => g.Select(
    (ff, relIndex) => new { ff, relIndex })))
.ToDictionary(x => x.ff.Id, x => x.relIndex);
var oldFieldsIndexesByType = CountFieldsByType(obsoleteTemplate);
var newFieldsIndexesByType = CountFieldsByType(submittedTemplate);
var oldFieldsFoundReplacements = obsoleteTemplate.Fields.ToDictionary(f => f.Id, f => false);
var newFieldsToAdd = new HashSet<TemplateField>();
foreach (var kv_new in newFieldsIndexesByType)
{
    var newField = submittedTemplate.Fields.First(sf => sf.Id == kv_new.Key);
    var oldField = obsoleteTemplate.Fields.FirstOrDefault(of =>
        of.Type.Id == newField.Type.Id &&
        oldFieldsIndexesByType.TryGetValue(of.Id, out var x) && x == kv_new.Value);
    bool FoundOldFieldToReuse = (oldField != null);
    if (FoundOldFieldToReuse)
    {
        oldFieldsFoundReplacements[oldField.Id] = true;
        oldField.Hidden = false;
        oldField.Title = newField.Title;
        oldField.Description = newField.Description;
        oldField.Position = newField.Position;
        oldField.ShowInAnalytics = newField.ShowInAnalytics;
    }
    else
    {
        var newFieldToAdd = new TemplateField()
        {
            Template = obsoleteTemplate,
            Hidden = false,
            Title = newField.Title,
            Description = newField.Description,
            Type = newField.Type,
            Position = newField.Position,
            ShowInAnalytics = newField.ShowInAnalytics,
        };
        newFieldsToAdd.Add(newFieldToAdd);
        obsoleteTemplate.Fields.Add(newFieldToAdd);
    }
}
foreach (var oldFieldToHideId in oldFieldsFoundReplacements.Where(kv => !kv.Value).Select(kv => kv.Key))
    obsoleteTemplate.Fields.First(of => of.Id == oldFieldToHideId).Hidden = true;

await _ctx.TemplateFields.AddRangeAsync(newFieldsToAdd);
obsoleteTemplate.Title = submittedTemplate.Title;
obsoleteTemplate.Description = submittedTemplate.Description;
obsoleteTemplate.Public = submittedTemplate.Public;
obsoleteTemplate.Topic = submittedTemplate.Topic;
obsoleteTemplate.AllowedUsers = submittedTemplate.AllowedUsers;
obsoleteTemplate.CoverImageUrl = submittedTemplate.CoverImageUrl;
try
{
    await _ctx.SaveChangesAsync();
}
catch (Exception)
{
    return null;
}
return obsoleteTemplate;
}

public async Task DeleteTemplate(int templateId)
{
    Template? template = await GetTemplate(templateId);
    if (template == null) return;
    if (!await TemplateAuthManage(template)) return;
    _ctx.Templates.Remove(template);
    await _ctx.SaveChangesAsync();
    // Soft delete not stated in requirements
}

public async Task LikeTemplate(int templateId, bool liked)
{
    var appUser = await GetCurrentUser();
    if (appUser == null) return;
    var template = await GetTemplate(templateId);
    if (template == null) return;
    var curUserAllowedToInteract = template.Public || await CurrentUserHasAdmin() || (!template.Public &&
template.AllowedUsers.Contains(appUser));
    if (!curUserAllowedToInteract) return;
    if (liked)
        template.UsersWhoLiked.Add(appUser);
}

```



```

        else
            template.UsersWhoLiked.Remove(appUser);
        await _ctx.SaveChangesAsync();
    }
    public async Task CommentOnTemplate(int templateId, string commentText)
    {
        if (string.IsNullOrWhiteSpace(commentText)) return;
        var appUser = await GetCurrentUser();
        if (appUser == null) return;
        var template = await GetTemplate(templateId);
        if (template == null) return;
        var curUserAllowedToInteract = await CurrentUserHasAdmin() || template.Public || (!template.Public &&
template.AllowedUsers.Contains(appUser));
        if (!curUserAllowedToInteract) return;
        template.Comments.Add(new TemplateComment
        {
            PostedAt = DateTime.UtcNow,
            User = appUser,
            Template = template,
            Text = commentText,
        });
        await _ctx.SaveChangesAsync();
    }
    public async Task<List<WoasFormsAppUser>> GetAllUsers()
    {
        return await _users.Users
            .Include(u => u.OwnedTemplates)
            .ToListAsync();
    }
    public async Task DeleteUser(string targetUserId)
    {
        if (!await CurrentUserHasPowerOverTarget(targetUserId)) return;
        var targetUser = await GetUserById(targetUserId);
        await _users.UpdateSecurityStampAsync(targetUser);
        if (await GetCurrentUser() == targetUser) await _sign.SignOutAsync();
        _ctx.Users.Remove(await GetUserById(targetUserId));
        await _ctx.SaveChangesAsync();
    }
    public async Task GiveUserRole(string userId, string roleName)
    {
        if (!await CurrentUserHasAdmin()) return;
        await _users.AddToRoleAsync(await GetUserById(userId), roleName);
        await _ctx.SaveChangesAsync();
    }
    public async Task RevokeUserRole(string userId, string roleName)
    {
        if (!await CurrentUserHasAdmin()) return;
        var targetUser = await GetUserById(userId);
        await _users.RemoveFromRoleAsync(targetUser, roleName);
        await _users.UpdateSecurityStampAsync(targetUser);
        if (roleName == "Admin" && await GetCurrentUser() == targetUser) await _sign.SignOutAsync();
        await _ctx.SaveChangesAsync();
    }
    public async Task BlockUser(string userId, bool blocked)
    {
        if (!await CurrentUserHasAdmin()) return;
        var targetUser = await GetUserById(userId);
        targetUser.IsBlocked = blocked;
        await _users.UpdateSecurityStampAsync(targetUser);
        await _ctx.SaveChangesAsync();
    }
    public async Task BlockUser(string userId) => await BlockUser(userId, true);
    public async Task UnblockUser(string userId) => await BlockUser(userId, false);
    public async Task<List<string>> GetUserRoles(string userId)
    {
        List<string> res = new List<string>();
        if (!await CurrentUserHasAdmin()) return res;
        res = (await _users.GetRolesAsync(await GetUserById(userId))).ToList();
        return res;
    }
    private static IQueryable<Response> GetResponseDetails(IQueryable<Response> query)
    {
        return query
            .Include(r => r.Respondent)
            .Include(r => r.Template)
            .ThenInclude(t => t.Owner)
            .Include(r => r.Template)
            .Include(r => r.Answers)
            .ThenInclude(a => a.Field)
            .ThenInclude(f => f.Type);
    }
}

```

```

private async Task<IQueryable<Response>> ResponseAuthVisibility(IQueryable<Response> query)
{
    var curUserHasAdmin = await CurrentUserHasAdmin();
    if (curUserHasAdmin) return query;
    var curUser = await GetCurrentUser();
    return query.Where(r => r.Respondent == curUser);
}
public async Task<bool> GetCurrentUserOwnsResponse(int responseId)
{
    if (await CurrentUserHasAdmin()) return true;
    var curUser = await GetCurrentUser();
    return curUser == (await GetResponse(responseId)).Respondent;
}
public async Task<IEnumerable<Response>> GetResponsesByTemplate(int templateId)
{
    var res = _ctx.Responses.Where(r => r.Template.Id == templateId);
    res = await ResponseAuthVisibility(res);
    res = GetResponseDetails(res);
    return res;
}
public async Task<IEnumerable<Response>> GetResponsesByRespondent(string userId)
{
    var res = _ctx.Responses.Where(r => r.Respondent.Id == userId);
    res = await ResponseAuthVisibility(res);
    res = GetResponseDetails(res);
    return res;
}
public async Task<IEnumerable<Response>> GetResponsesByCurrentUser()
{
    var curUser = await GetCurrentUser();
    var res = _ctx.Responses.Where(r => r.Respondent.Id == curUser.Id);
    //res = await ResponseAuthVisibility(res);
    res = GetResponseDetails(res);
    return res;
}
public async Task<Response?> GetResponse(int responseId)
{
    var res = _ctx.Responses.Where(r => r.Id == responseId);
    res = await ResponseAuthVisibility(res);
    res = GetResponseDetails(res);
    return await res.FirstAsync();
}
public async Task<IEnumerable<Response>> GetAllResponses()
{
    var res = _ctx.Responses.AsQueryable();
    res = await ResponseAuthVisibility(res);
    res = GetResponseDetails(res);
    return res;
}
public async Task<Response?> CreateResponse(Response response)
{
    var curUser = await GetCurrentUser();
    response.Respondent = curUser;
    response.CreationDate = DateTime.UtcNow;
    var freshResponse = await _ctx.Responses.AddAsync(response);
    return freshResponse.Entity;
}
public async Task<WoasFormsAppUser?> GetUser(string userId)
=> await GetUserById(userId);
public async IAsyncEnumerable<WoasFormsAppUser> GetUsers(IEnumerable<string> userIds)
{
    foreach (var userId in userIds)
    {
        var user = await GetUserById(userId);
        if (user != null) yield return user;
    }
}
public async Task<Dictionary<TemplateTag, int>> GetTemplateCountsByTag()
{
    var res = await _ctx.Templates.Include(t => t.Tags).SelectMany(t => t.Tags).GroupBy(x => x).ToDictionaryAsync(g => g.Key, g =>
g.Count());
    return res;
}
}
}

```

Реализация сервиса JiraTicketsService

```

namespace WoasFormsApp.Services
{
    public class JiraTicketsService(
        IHttpClientFactory factory,
        IDatabaseAccessorService dba,
        WoasFormsDbContext ctx) : IJiraTicketsService
    {
        private HttpClient GetClient() => factory.CreateClient("jira");
        private HttpClient GetClientDesk() => factory.CreateClient("jira_servicedesk");
        public record CustomerCreateResponse (string accountId);
        public async Task<JiraCustomerView?> CreateCustomerFromCurrentUser(string? overrideEmail = null)
        {
            var client = GetClientDesk();
            var curUser = await dba.GetCurrentUser();
            if (curUser == null) return null;
            var request = new HttpRequestMessage(HttpMethod.Post, "customer/");
            request.Content = JsonContent.Create(
                new
                {
                    displayName = curUser.UserName,
                    email = overrideEmail ?? curUser.Email,
                }
            );
            var response = await client.SendAsync(request);
            try { response.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
            var responseRead = await response.Content.ReadFromJsonAsync<CustomerCreateResponse>();
            curUser.JiraAccountId = responseRead.accountId;
            await ctx.SaveChangesAsync();
            return await GetCustomerById(responseRead.accountId);
        }
        public record IssueGetResponse(
            string id,
            IssueGetResponseFields fields
        );
        public record IssueGetResponseFields(
            string summary,
            IssueGetResponseStatus status,
            IssueGetResponsePriority priority,
            IssueGetResponseReporter reporter
        );
        public record IssueGetResponseStatus(string name);
        public record IssueGetResponsePriority(string name);
        public record IssueGetResponseReporter(string accountId);
        public async Task<JiraTicketView?> GetTicketById(string ticketId)
        {
            var client = GetClient();
            var ticketRequest = new HttpRequestMessage(HttpMethod.Get, $"issue/{ticketId}");
            var ticketResponse = await client.SendAsync(ticketRequest);
            try { ticketResponse.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
            var ticketResponseRead = await ticketResponse.Content.ReadFromJsonAsync<IssueGetResponse>();
            if (ticketResponseRead == null) return null;
            return new JiraTicketView
            {
                Id = ticketId,
                Status = ticketResponseRead.fields.status.name,
                Priority = ticketResponseRead.fields.priority.name,
                Summary = ticketResponseRead.fields.summary,
            };
        }
        public record SearchIssueResponseItem(string id);
        public record SearchIssuesByUserResponse(List<SearchIssueResponseItem> issues);
        public record FetchIssuesResponse(IssueGetResponse[] issues);
        private async Task<List<JiraTicketView>> GetTicketsByUser(string userId)
        {
            var res = new List<JiraTicketView>();
        }
    }
}

```

```

var client = GetClient();
var request = new HttpRequestMessage(HttpMethod.Get, $"search/jql?jql=reporter={userId}");
var response = await client.SendAsync(request);
try { response.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
var responseRead = await response.Content.ReadFromJsonAsync<SearchIssuesByUserResponse>();
responseRead ??= new SearchIssuesByUserResponse(new());
var ticketIds = responseRead.issues.Select(i => i.id);
var requestFetch = new HttpRequestMessage(HttpMethod.Post, $"issue/bulkfetch");
requestFetch.Content = JsonContent.Create(new {
    expand = new[] { "names" },
    fields = new[] { "summary", "status", "priority" },
    issueIdsOrKeys = ticketIds.ToArray(),
});
var responseFetch = await client.SendAsync(requestFetch);
try { responseFetch.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
var responseFetchRead = await responseFetch.Content.ReadFromJsonAsync<FetchIssuesResponse>();
res.AddRange(responseFetchRead.issues.Select(fetched => new JiraTicketView
{
    Id = fetched.id,
    Status = fetched.fields.status.name,
    Priority = fetched.fields.priority.name,
    Summary = fetched.fields.summary,
}));
return res;
}
public record JiraCustomerGetResponse(
    string accountId,
    string displayName,
    string emailAddress
);
private async Task<JiraCustomerView?> GetCustomerByEmail(string email)
{
    var client = GetClient();
    var request = new HttpRequestMessage(HttpMethod.Get, $"user/search/?query={email}");
    var response = await client.SendAsync(request);
    try { response.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
    var responseRead = await response.Content.ReadFromJsonAsync<JiraCustomerGetResponse[]>();
    if (responseRead == null || responseRead.Count() == 0) return null;
    var foundUser = responseRead.First();
    var tickets = await GetTicketsByUser(foundUser.accountId);
    var result = new JiraCustomerView
    {
        AccountId = foundUser.accountId,
        DisplayName = foundUser.displayName,
        Email = foundUser.emailAddress,
        Tickets = tickets,
    };
    return result;
}
private async Task<JiraCustomerView?> GetCustomerById(string userId)
{
    var client = GetClient();
    var request = new HttpRequestMessage(HttpMethod.Get, $"user/?accountId={userId}");
    var response = await client.SendAsync(request);
    try { response.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
    var responseRead = await response.Content.ReadFromJsonAsync<JiraCustomerGetResponse>();
    if (responseRead == null) return null;
    var tickets = await GetTicketsByUser(responseRead.accountId);
    var result = new JiraCustomerView
    {
        AccountId = responseRead.accountId,
        DisplayName = responseRead.displayName,
        Email = responseRead.emailAddress,
        Tickets = tickets,
    };
    return result;
}
public async Task<JiraCustomerView?> GetCustomerFromCurrentUser()
{
    var curUser = await dba.GetCurrentUser();
    if (curUser == null) return null;
}

```

```

        if (!await CreateCustomerIfNeeded()) return null;
        ctx.Update(curUser);
        return await GetCustomerById(curUser.JiraAccountId);
    }
    private async Task<bool> CreateCustomerIfNeeded(string? overrideEmail = null)
    {
        var curUser = await dba.GetCurrentUser();
        if (curUser == null) return false;
        if (string.IsNullOrWhiteSpace(curUser.JiraAccountId))
        {
            var foundCustomer = await GetCustomerByEmail(curUser.Email?? "");
            if (foundCustomer != null)
            {
                curUser.JiraAccountId = foundCustomer.AccountId;
                await ctx.SaveChangesAsync();
            }
            else
            {
                var newSupportCustomer = await CreateCustomerFromCurrentUser(overrideEmail);
                if (newSupportCustomer == null) return false;
            }
        }
        return true;
    }
    public record IssueCreateResponse(string id);
    public async Task<JiraTicketView?> CreateTicketByCurrentUser(NewTicketModel model)
    {
        await CreateCustomerIfNeeded(overrideEmail: model.Email);
        var curUser = await dba.GetCurrentUser();
        if (curUser == null) return null;
        var reporterId = curUser.JiraAccountId;
        var payload = new
        {
            fields = new
            {
                project = new { key = "SUP" },
                issuetype = new { id = "10006" },
                summary = model.Summary,
                reporter = new { id = reporterId }
            }
        };
        var client = GetClient();
        var request = new HttpRequestMessage(HttpMethod.Post, "issue");
        request.Content = JsonContent.Create(payload);
        var response = await client.SendAsync(request);
        try { response.EnsureSuccessStatusCode(); } catch (Exception) { return null; }
        var newTicketId = (await response.Content.ReadFromJsonAsync<IssueCreateResponse>())!.id;
        return await GetTicketById(newTicketId);
    }
}
}
}

```

Реализация сервиса SFContactManager

```

namespace WoasFormsApp.Services
{
    public class SFContactManager(
        IHttpClientFactory factory,
        IConfiguration config,
        WoasFormsDbContext ctx,
        ICientCredentialsTokenManagementService tokenManager,
        IDatabaseAccessorService dba) : ISFContactManager
    {
        private string _ver = "v63.0";
        private async Task<HttpClient> GetClient()
        {
            var client = factory.CreateClient("salesforce");
            var tokenReq = await tokenManager.GetAccessTokenAsync("salesforce");
            var token = tokenReq.AccessToken;
            client.SetBearerToken(token ?? "");
            return client;
        }
        public class CreateResponse
        {
            public class ResponseItem
            {
                {
                    public class Body
                    {
                        public string id { get; set; }
                        public bool success { get; set; }
                    }
                    public Body body { get; set; }
                    public required int httpStatusCode { get; set; }
                    public required string referenceId { get; set; }
                }
            }
            public List<ResponseItem> compositeResponse { get; set; }
        }
        public async Task<sfUserDataView?> CreateCurrentUserSFContact()
        {
            var curUser = await dba.GetCurrentUser();
            if (curUser == null) return null;
            var request = new HttpRequestMessage(HttpMethod.Post, $"/services/data/{_ver}/composite");
            var actions = new List<dynamic>();
            actions.Add(new
            {
                method = "POST",
                url = $"/services/data/{_ver}/subjects/Account",
                referenceId = "NewAccount",
                body = new
                {
                    Name = curUser.UserName
                }
            });
            actions.Add(new
            {
                method = "POST",
                url = $"/services/data/{_ver}/subjects/Contact",
                referenceId = "NewContact",
                body = new
                {
                    AccountId = "@{NewAccount.id}",
                    LastName = curUser.UserName ?? "",
                    Email = curUser.Email ?? ""
                }
            });
            var contentJson = new
            {
                allOrNone = true,
                compositeRequest = actions.ToArray(),
            }
        }
    }
}

```

```

    };
    request.Content = JsonConvert.Create(contentJson);
    var Client = await GetClient();
    var response = await Client.SendAsync(request);
    try {response.EnsureSuccessStatusCode();} catch (Exception) { return null;}
    var responseRead = await response.Content.ReadFromJsonAsync<CreateResponse>();
    curUser.SalesForceAccountId = responseRead.compositeResponse.First(x => x.referenceId ==
    "NewAccount").body.id;
    curUser.SalesForceContactId = responseRead.compositeResponse.First(x => x.referenceId ==
    "NewContact").body.id;
    await ctx.SaveChangesAsync();
    return await GetUserSFContact(curUser.SalesForceContactId);
}
public class GetContactResponse
{
    public string? Email { get; set; }
    public string Name { get; set; }
    public string? FirstName { get; set; }
    public string LastName { get; set; }
    public string? Description { get; set; }
    public DateOnly? Birthdate { get; set; }
}
private async Task<sfUserDataView?> GetUserSFContact(string id)
{
    var Client = await GetClient();
    var response = await Client.GetAsync($" /services/data/{_ver}/subjects/Contact/{id}");
    if (!response.IsSuccessStatusCode) return null;
    var responseData = await response.Content.ReadFromJsonAsync<GetContactResponse>();
    sfUserDataView res = new sfUserDataView
    {
        SalesForceContactID = id,
        FirstName = responseData.FirstName,
        LastName = responseData.LastName,
        Email = responseData.Email ?? "",
        About = responseData.Description ?? "",
        BirthDay = responseData.Birthdate,
    };
    return res;
}
public async Task<sfUserDataView?> GetCurrentUserSFContact()
{
    var curUser = await dba.GetCurrentUser();
    if (curUser == null || string.IsNullOrEmpty(curUser.SalesForceContactId)) return null;
    return await GetUserSFContact(curUser.SalesForceContactId);
}
public async Task<bool> UpdateCurrentUserSFContact(sfUserDataView view)
{
    var Client = await GetClient();
    var request = new HttpRequestMessage(HttpMethod.Patch,
    $" /services/data/{_ver}/subjects/Contact/{view.SalesForceContactID}");
    var contentDict = new
    {
        view.Email,
        Description = view.About,
        Birthdate = view.BirthDay,
        view.FirstName,
        view.LastName,
    };
    request.Content = JsonConvert.Create(contentDict);
    var response = await Client.SendAsync(request);
    return response.StatusCode == System.Net.HttpStatusCode.NoContent;
}
}
}

```

ПОСЛЕДНИЙ ЛИСТ ВКР

Выпускная квалификационная работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

«23» июня 2025 г.

_____ (Коротких Д.А.)
(подпись) (Ф.И.О)